



CNN FOR SIGNAL CLASSIFICATION AND GCNN FOR VLSI TIMING PREDICTION

A SUMMER INTERNSHIP REPORT

Submitted by

KRISH BHAVSAR

(230130111040)

In partial fulfillment of the academic requirements of the
7th Semester of

BACHELOR OF ENGINEERING

in

Electronics and Communication Engineering

Carried out at

Indian Institute of Information Technology Vadodara

Government Engineering College, Sector 28, Gandhinagar

Gujarat Technological University, Ahmedabad

July, 2026

Government Engineering College, Sector 28, Gandhinagar

Department of Electronics and Communication Engineering

CERTIFICATE

This is to certify that the Summer Internship work embodied in this report entitled **"CNN for Signal Classification and GCNN for VLSI Timing Prediction"** was carried out by **Krish Bhavsar (230130111040)** at the **Indian Institute of Information Technology Vadodara**, in partial fulfilment of the requirements of the subject Summer Internship (3170001) of the 7th Semester of the Bachelor of Engineering (B.E.) degree in Electronics and Communication Engineering of Gujarat Technological University. This internship work has been carried out under my supervision and is found to be satisfactory to the best of my knowledge and to the satisfaction of the department.

Prof. Jignesh K. Bhavsar	**Dr. T. B. Sheth**
Internal Guide	Head of the Department

COMPLETION CERTIFICATE

Certificate of completion of the internship issued by the Indian Institute of Information Technology Vadodara, for the period 03/07/2026 to 16/07/2026.

[The signed completion certificate on the letterhead of IIIT Vadodara is to be attached here.]

Indian Institute of Information Technology Vadodara
Block No. 9, c/o Government Engineering College Campus,
Sector 28, Gandhinagar – 382028, Gujarat.
Phone No.: 079 - 23977 501 | URL: www.iiitvadodara.ac.in

No: Date: 16/07/2026

TO WHOM IT MAY CONCERN

This is to certify that, Mr. Krish Bhavsar Enrollment No 230130111040, Student of Government Engineering College, Gandhinagar (Gujarat Technological University), B.E. Electronics and Communication Engineering, has successfully completed a two week Internship in the field of VLSI Machine Learning for Electronic Design Automation (EDA), based on Graph Convolutional Neural Networks (GCNN) for Circuit Netlist Representation and Design-Quality Prediction during the period of 03/07/2026 to 16/07/2026.

During the period of his internship program with us, he had been exposed to different modelling, experimentation and analysis processes, and was found sincere, dedicated and hardworking.

He carried out this work as a Research Intern at the Indian Institute of Information Technology, Vadodara (IIIT Vadodara), under the guidance of Dr. Kamal Kishor Jha. We wish him the very best for his future endeavours.

Authorised Signature Mentor Signature

Institute Head

Government Engineering College, Sector 28, Gandhinagar

DECLARATION

I hereby declare that the internship report entitled **"CNN for Signal Classification and GCNN for VLSI Timing Prediction"**, submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Electronics and Communication Engineering to Gujarat Technological University, Ahmedabad, is a bonafide record of the original work carried out by me during my summer internship at the Indian Institute of Information Technology Vadodara, from 3 July 2026 to 16 July 2026, under the supervision of Dr. Kamal Kishor Jha and the guidance of my institute guide Prof. Jignesh K. Bhavsar.

I further declare that no part of this report has been directly copied from any student's report or taken from any other source without providing due reference.

Date: _____

Place: Gandhinagar

Name of the Student

Krish Bhavsar (230130111040)

Sign of the Student

_____

ACKNOWLEDGEMENT

I am grateful to everyone who helped me to complete my summer internship on "CNN for Signal Classification and GCNN for VLSI Timing Prediction", carried out from 3 July 2026 to 16 July 2026.

I sincerely thank the Indian Institute of Information Technology Vadodara for giving me the opportunity to work as a research intern. I express my deep gratitude to my supervisor, Dr. Kamal Kishor Jha, for accepting me as an intern, for defining the problem, for agreeing to change its direction when the results called for it, and for his patient guidance and careful review of the work at every stage. His insistence that every number be reproducible shaped this work.

I express my heartfelt thanks to my internal guide, Prof. Jignesh K. Bhavsar, for his guidance, encouragement and support throughout the internship and the preparation of this report. I am equally thankful to Dr. T. B. Sheth, Head of the Department of Electronics and Communication Engineering, to the faculty coordinators of the summer internship, and to all the faculty members of the department for their support.

I also thank Government Engineering College, Sector 28, Gandhinagar, and Gujarat Technological University for including the summer internship in the curriculum and giving students an early exposure to work outside the classroom.

Finally, I thank my family and my friends for their constant support.

Krish Bhavsar

Enrolment No.: 230130111040

ABSTRACT

This report describes the summer internship carried out at the Indian Institute of Information Technology Vadodara under Dr. Kamal Kishor Jha. For the GTU subject Summer Internship (3170001) the period considered is 3 July 2026 to 16 July 2026; the report describes the complete work of the research internship, which applied deep learning first to a biomedical signal and then to the timing of digital circuits.

A one-dimensional squeeze-and-excitation residual network classified the 8,528 single-lead recordings of the PhysioNet/CinC 2017 dataset into four rhythm classes with 80.5 % accuracy and a macro F1 of 0.730. The same recordings were converted into spectrogram, scalogram and DWT images and classified by eighteen two-dimensional networks and two residual translations; the best reached a macro F1 of 0.660, below the one-dimensional network, and the work was re-scoped.

The main phase built CalibTimer, a graph neural network with 40,730 parameters that predicts static timing at every pin of a placed but unrouted design. Twenty-seven sky130 designs were taken through the OpenROAD flow and parsed from their Verilog, DEF, SPEF, SDF, SDC and Liberty files into pin-level graphs. The network propagates arrival time level by level, as a timing engine does, and reads gate delays from the Liberty tables through a learned lookup. Trained on fifteen designs and tested on seven unseen ones, it reached a mean arrival-time R^2 of 0.8988, with 94 to 99 % of the pins of every design within 0.25 ns; a whole design is evaluated in 8 to 160 ms. Split-conformal calibration on held-out designs gave a slack interval of ± 2.35 ns with a measured coverage of 96.5 to 100 % against 90 % required.

Twelve tests were run against the code and the trained model. They reproduced the reported results exactly and found that the input-delay feature had been zero for every design because of a parsing defect, which explains the weakest test result; a corrected parser reads all 4,317 input-delay constraints.

LIST OF FIGURES

Fig 2.1 Flow of the work carried out during the internship 4
Fig 2.2 Work carried out in the two weeks of the GTU period 5
Fig 3.1 Ten seconds of each of the four rhythm classes of the PhysioNet/CinC 2017 dataset 6
Fig 3.2 Arrival time, required arrival time and slack on a timing path 8
Fig 3.3 Position of the pre-routing prediction in the OpenROAD flow 8
Fig 4.1 Size of the 27 benchmark designs (logic cells, log scale) 13
Fig 4.2 Real placed layouts of two test designs, drawn from their DEF files 14
Fig 5.1 Class distribution of the dataset 15
Fig 5.2 Architecture of the one-dimensional SE-ResNet 16
Fig 5.3 Training history of the one-dimensional SE-ResNet 17
Fig 6.1 From a one-dimensional signal to a two-dimensional image 19
Fig 6.2 One recording as a spectrogram, a scalogram and a DWT image 20
Fig 6.3 Test macro F1 of the 18 networks of the architecture sweep 20
Fig 6.4 Architecture of the two-dimensional SE-ResNet 21
Fig 6.5 Accuracy and macro F1 of the one- and two-dimensional models 22
Fig 6.6 Confusion matrices: 1-D SE-ResNet and a 2-D CNN on scalograms 23
Fig 7.1 From the design files to a trainable graph 24
Fig 7.2 The heterogeneous pin-level timing graph (schematic) 25
Fig 7.3 The real timing graph of seq_detector, built by the parser 26
Fig 7.4 A real Liberty delay table (sky130_fd_sc_hd__nand2_1, A to Y) 27
Fig 7.5 Division of the 27 designs into training, test and calibration groups 28
Fig 8.1 Block diagram of the CalibTimer network 29
Fig 8.2 The CellLUTModule: a learned read of the Liberty table 30
Fig 8.3 Level-ordered propagation of arrival time 31
Fig 8.4 Per-design effect of the candidate fixes 33
Fig 9.1 Training curve of the final model 34
Fig 9.2 Arrival-time R^2 on each unseen test design 35
Fig 9.3 Predicted against true arrival time for three test designs 36
Fig 9.4 Distribution of the arrival-time error on each test design 36
Fig 9.5 Arrival time on the real die of usbf_device: true, predicted and error 37
Fig 9.6 The failures against the input delay, and the arrival time predicted at the input pins 38
Fig 9.7 The split-conformal calibration procedure 39
Fig 9.8 Measured coverage of the calibrated interval on each test design 40
Fig A1.1 No Objection Certificate issued by the EC Department 47
Fig A2.1 Selection email and acceptance 48

LIST OF TABLES

Table 2.1 Week-wise schedule of the GTU summer-internship period 5
Table 3.1 Summary of the papers studied 9
Table 4.1 Software used 11
Table 4.2 Hardware used 11
Table 4.3 The 27 benchmark designs (read from their DEF and SDC files) 12
Table 5.1 Classes of the PhysioNet/CinC 2017 dataset 15
Table 5.2 Training settings of the one-dimensional networks 17
Table 5.3 Results of the one-dimensional networks on the test set 17
Table 5.4 Per-class results of the one-dimensional SE-ResNet on the test set 18
Table 6.1 The three time-frequency representations 19
Table 6.2 Kernel-size ablation (simple two-dimensional CNN) 21
Table 6.3 Best results of the two-dimensional networks on the test set 21
Table 6.4 Per-class results of the two-dimensional SE-ResNet on the test set 22
Table 6.5 Best model of each phase 23
Table 7.1 Files read for each design 24
Table 7.2 The twelve node features 26
Table 7.3 The three groups of designs 28
Table 8.1 Training configuration of the final model 32
Table 8.2 Experiments: mean arrival-time R^2 on the seven test designs 32
Table 9.1 Results on the seven unseen test designs 35
Table 9.2 Arrival time at the primary-input pins (medians) and the overall error bias 38
Table 9.3 Measured coverage of the calibrated slack interval (± 2.3499 ns) 40
Table 9.4 Time to predict a whole design, against routing and STA 41
Table 9.5 Arrival-time R^2 on the seven test designs: this work and the reference papers 42
Table 9.6 Test cases 43
Table 10.1 Problems encountered and solutions 45

LIST OF ABBREVIATIONS

Abbreviation	Full form	Abbreviation	Full form
AF	Atrial Fibrillation	LUT	Look-Up Table
AT	Arrival Time	MAE	Mean Absolute Error
BN	Batch Normalisation	MLP	Multi-Layer Perceptron
CNN	Convolutional Neural Network	MSE	Mean Squared Error
CTS	Clock Tree Synthesis	NOC	No Objection Certificate
CWT	Continuous Wavelet Transform	ORFS	OpenROAD Flow Scripts
DAC	Design Automation Conference	PDK	Process Design Kit
DAG	Directed Acyclic Graph	PI	Primary Input
DEF	Design Exchange Format	PMMS	Project Management and Monitoring System (GTU)
DWT	Discrete Wavelet Transform	RAT	Required Arrival Time
ECG	Electrocardiogram	ReLU	Rectified Linear Unit
EDA	Electronic Design Automation	SDC	Synopsys Design Constraints
GCNN	Graph Convolutional Neural Network	SDF	Standard Delay Format
GNN	Graph Neural Network	SE	Squeeze-and-Excitation
GPU	Graphics Processing Unit	SPEF	Standard Parasitic Exchange Format
GTU	Gujarat Technological University	STA	Static Timing Analysis
IIIT	Indian Institute of Information Technology	STFT	Short-Time Fourier Transform
LEF	Library Exchange Format	VLSI	Very Large Scale Integration

TABLE OF CONTENTS

Declaration i
Acknowledgement ii
Abstract iii
Chapter 1: Overview of the Organisation 1
1.1 About IIIT Vadodara 1
1.2 Research Internship Programme 1
1.3 Supervisor and Area of Work 1
1.4 Facilities Used During the Internship 2
Chapter 2: Introduction to the Internship 3
2.1 Internship Summary 3
2.2 Purpose 3
2.3 Objectives 3
2.4 Scope 4
2.5 Flow of the Work 4
2.6 Schedule 5
Chapter 3: Literature Review 6
3.1 The ECG Signal and Arrhythmia 6
3.2 Convolutional Neural Networks 6
3.3 Graph Neural Networks 7
3.4 Static Timing Analysis 7
3.5 Papers Studied 8
3.6 Gap Identified 10
Chapter 4: Tools and Environment 11
4.1 Software Environment 11
4.2 Hardware 11
4.3 The OpenROAD Flow and the sky130 Library 11
4.4 The Benchmark Designs 12
Chapter 5: Phase 1: One-Dimensional CNN on ECG 15
5.1 Problem Statement 15
5.2 The Dataset 15
5.3 Preprocessing 16
5.4 Network Architecture 16
5.5 Training 16
5.6 Results of Phase 1 17
Chapter 6: Phase 2: Two-Dimensional CNN on Time-Frequency Images 19
6.1 Motivation 19

6.2 Generating the Images 19
6.3 Architecture Sweep and Kernel Ablation 20
6.4 Residual Networks 21
6.5 Results of Phase 2 21
6.6 Why the Approach Was Discontinued 22
Chapter 7: Phase 3: CalibTimer — Data and Graph Construction 24
7.1 Overview 24
7.2 Design Files and Parser 24
7.3 Graph Construction 25
7.4 Node and Edge Features 26
7.5 Division of the Designs 27
Chapter 8: Phase 3: CalibTimer — Model and Training 29
8.1 Network Architecture 29
8.2 Reading the Liberty Tables 29
8.3 Level-Ordered Propagation 30
8.4 Loss Function and Training 31
8.5 Experiments 32
Chapter 9: Results and Testing 34
9.1 Measures Used 34
9.2 Training Behaviour 34
9.3 Accuracy on the Unseen Test Designs 35
9.4 The Failure Case and What Was Done 37
9.5 Conformal Calibration of the Slack Prediction 39
9.6 Speed 41
9.7 Comparison with Reference Papers 41
9.8 Testing 42
Chapter 10: Conclusion and Discussion 44
10.1 Overall Analysis of the Internship 44
10.2 Main Results 44
10.3 Environment and Support at IIIT Vadodara 44
10.4 Problems Encountered and Their Solutions 45
10.5 Skills and Learning Outcomes 45
10.6 Limitations 46
10.7 Future Scope 46
10.8 Summary 46
Appendix 1: No Objection Certificate from the Department 47
Appendix 2: Selection (Joining) Email from IIIT Vadodara 48
Appendix 3: Weekly Record of Internship (Annexure 1) 49

Appendix 4: Selected Code Listings.....55
References.....57

CHAPTER 1: OVERVIEW OF THE ORGANISATION

1.1 ABOUT IIIT VADODARA

The Indian Institute of Information Technology Vadodara (IIIT Vadodara) was established in 2013 under the public-private partnership model of the Government of India, with the Government of India, the Government of Gujarat and industry partners as stakeholders. It is an Institute of National Importance under the Indian Institutes of Information Technology (Public-Private Partnership) Act, 2017, and offers undergraduate, postgraduate and doctoral programmes. The internship was carried out at its Gandhinagar campus, which made it possible to attend the institute in person and to meet the supervisor regularly.

1.2 RESEARCH INTERNSHIP PROGRAMME

IIIT Vadodara runs a research internship programme in which students of other institutes work on a research problem under a faculty member. I applied to this programme and on 15 June 2026 the internship coordinator informed me that I had been selected for a research internship under the supervision of Dr. Kamal Kishor Jha; I accepted the offer on 16 June 2026. The selection email, which describes the research internship as one of at least two months, is reproduced in Appendix 2.

For the GTU subject Summer Internship (3170001) the period considered in this report is 3 July 2026 to 16 July 2026, the period for which the Department of Electronics and Communication Engineering issued its No Objection Certificate (Appendix 1). The research itself did not start or stop on those dates. The ECG work had begun after selection in June, and the timing-prediction work that began in the second GTU week continued under the research internship afterwards. This report describes the complete work, and Table 2.1 records what was done in each of the two GTU weeks.

1.3 SUPERVISOR AND AREA OF WORK

Dr. Kamal Kishor Jha is an Assistant Professor in the Department of Electronics and Communication Engineering at IIIT Vadodara, where he has served since 2015. He holds a Ph.D. and an M.Tech. in VLSI Design from the Indian Institute of Information Technology and Management, Gwalior. His research covers low-power VLSI design, nanoelectronics,

computer architecture, hardware accelerators for deep neural networks, and machine learning for chip design.

The internship sat at the meeting point of two of those interests. It began with neural networks for a biomedical signal, the kind of model such accelerators are built to run, and moved to a neural network that helps design the chip itself, by predicting the timing of a digital circuit before the expensive routing step. The change of direction was decided jointly with the supervisor, for the reasons given in Chapter 6.

1.4 FACILITIES USED DURING THE INTERNSHIP

- A personal laptop with an NVIDIA GeForce RTX 3050 6 GB graphics processor, used for every training run reported here.
- The Windows Subsystem for Linux, in which the OpenROAD physical design flow and the OpenSTA timing engine were run.
- Open-source software throughout: Python, PyTorch, TensorFlow/Keras, SciPy, PyWavelets, the OpenROAD Flow Scripts and the SkyWater sky130 process design kit.
- Regular review meetings with the supervisor, in which results were checked and the next task was decided.

CHAPTER 2: INTRODUCTION TO THE INTERNSHIP

2.1 INTERNSHIP SUMMARY

I completed the summer internship required by the subject Summer Internship (3170001) of the seventh semester of the B.E. programme of Gujarat Technological University at the Indian Institute of Information Technology Vadodara. The internship was carried out offline, and the work of the two GTU weeks is entered in the weekly record (Annexure 1, Appendix 3).

The title of the work is “CNN for Signal Classification and GCNN for VLSI Timing Prediction”. It has three phases. A one-dimensional convolutional neural network was first built to classify single-lead electrocardiogram recordings. The same recordings were then converted into time-frequency images and classified with two-dimensional networks, to test whether that route is better; it was not, and the work was re-scoped. The third and main phase built CalibTimer, a graph neural network that predicts the static timing of a digital integrated circuit at every pin after placement but before routing.

2.2 PURPOSE

Timing closure is one of the most expensive parts of chip design. The timing of a design is known accurately only after its wires have been routed, and routing a large design can take an hour or more. A model that predicts post-routing timing directly from the placed design would let a designer compare placements in seconds. The purpose of the internship was to learn how such a model is designed, trained on real industrial data and evaluated honestly, starting from the more familiar ground of convolutional networks on signals.

2.3 OBJECTIVES

- To study convolutional neural networks and apply them to the classification of single-lead electrocardiogram recordings.
- To find out experimentally whether time-frequency images classified by a two-dimensional network do better than a one-dimensional network on the raw signal.
- To study the physical design flow of digital circuits, static timing analysis, and the file formats in which a placed design and its timing are described.

- To design, implement and train a graph neural network that predicts static timing results at every pin of a placed, unrouted design.
- To evaluate that network on benchmark designs it has never seen, attach a statistically valid interval to its slack predictions, and test the whole pipeline.

2.4 SCOPE

The work covers data preparation, model design, training, evaluation, calibration and testing in software. The benchmark designs were taken through the open-source OpenROAD flow to obtain the input files and the reference timing; the flow was used as a tool and was not modified. No silicon was fabricated and no commercial electronic design automation tool was used. Clinical use of the ECG models is outside the scope of the work.

2.5 FLOW OF THE WORK

The order in which the work moved is shown in Fig 2.1. Each step was taken because of what the previous one showed: the two-dimensional route was a fair test of a popular idea and lost to the one-dimensional network, the timing problem called for a graph rather than an image, and the testing at the end of the work found a defect that explains one of the two designs on which the model falls short.

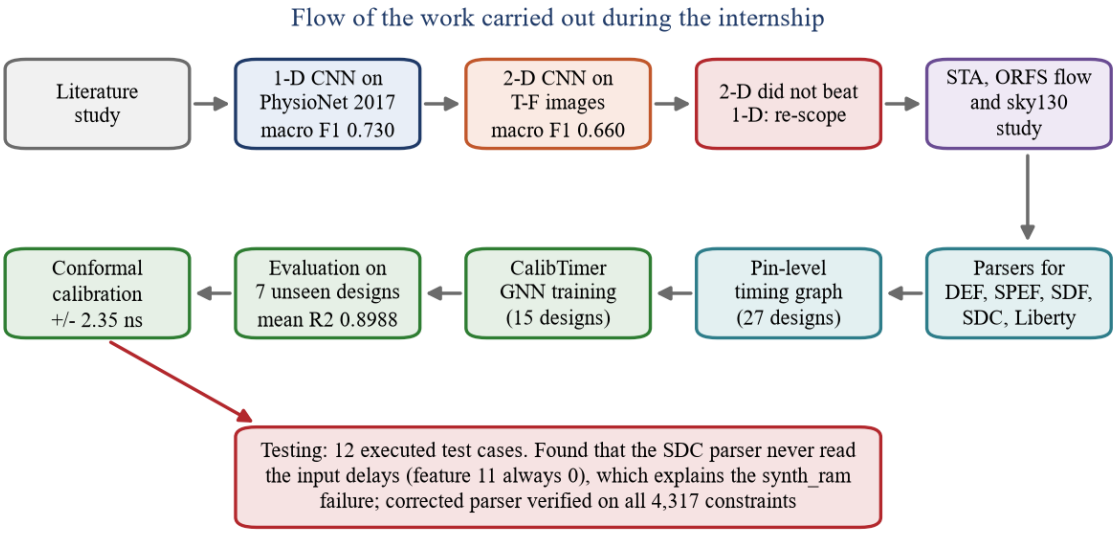


Fig 2.1 Flow of the work carried out during the internship

2.6 SCHEDULE

The work carried out in each of the two GTU weeks is given in Table 2.1 and Fig 2.2. The dates follow the time-stamps of the project files.

Table 2.1 Week-wise schedule of the GTU summer-internship period

Week	Dates	Work carried out	Chapter
1	03/07/2026 to 09/07/2026	Consolidation of the one-dimensional ECG results and a study of how the network could be deployed on hardware; regeneration of spectrogram, scalogram and DWT images for all 8,528 recordings on one common split; the 18-run two-dimensional architecture sweep and the kernel-size ablation; first reading on machine learning for timing	5, 6
2	10/07/2026 to 16/07/2026	Two-dimensional SE-ResNet and residual baseline; comparison with the one-dimensional network and the decision to re-scope; literature study on graph neural networks for timing prediction and on static timing analysis; set-up of the environment, study of the DEF, SPEF, SDF, SDC and Liberty formats, first parsers and the first graph neural network training runs on real designs	3, 4, 7

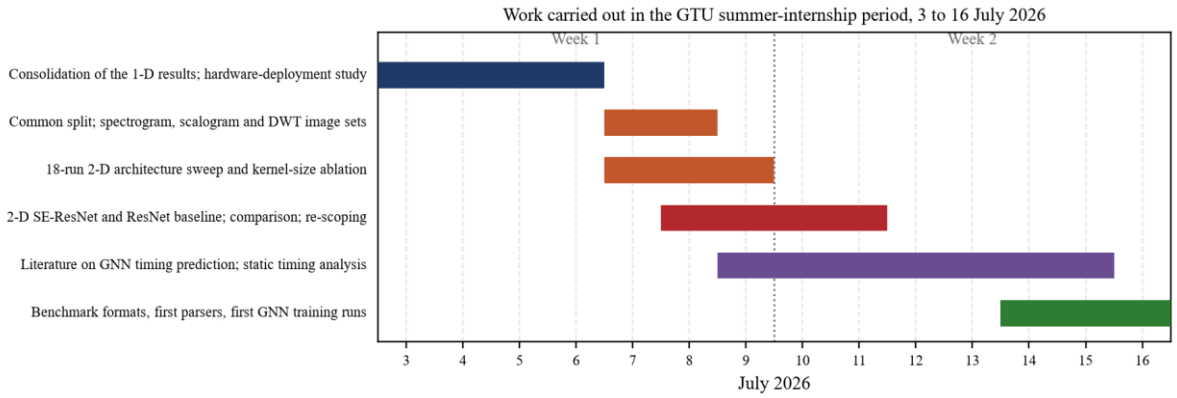


Fig 2.2 Work carried out in the two weeks of the GTU period

CHAPTER 3: LITERATURE REVIEW

3.1 THE ECG SIGNAL AND ARRHYTHMIA

The electrocardiogram records the electrical activity of the heart. One normal beat produces a P wave, from the depolarisation of the atria, a QRS complex, from the ventricles, and a T wave, from ventricular repolarisation. The interval between successive R peaks is the RR interval. Atrial fibrillation, the arrhythmia of most interest in this dataset, shows as the absence of a clear P wave and an irregular RR interval, so both the shape of the beat and the rhythm carry information (Fig 3.1).

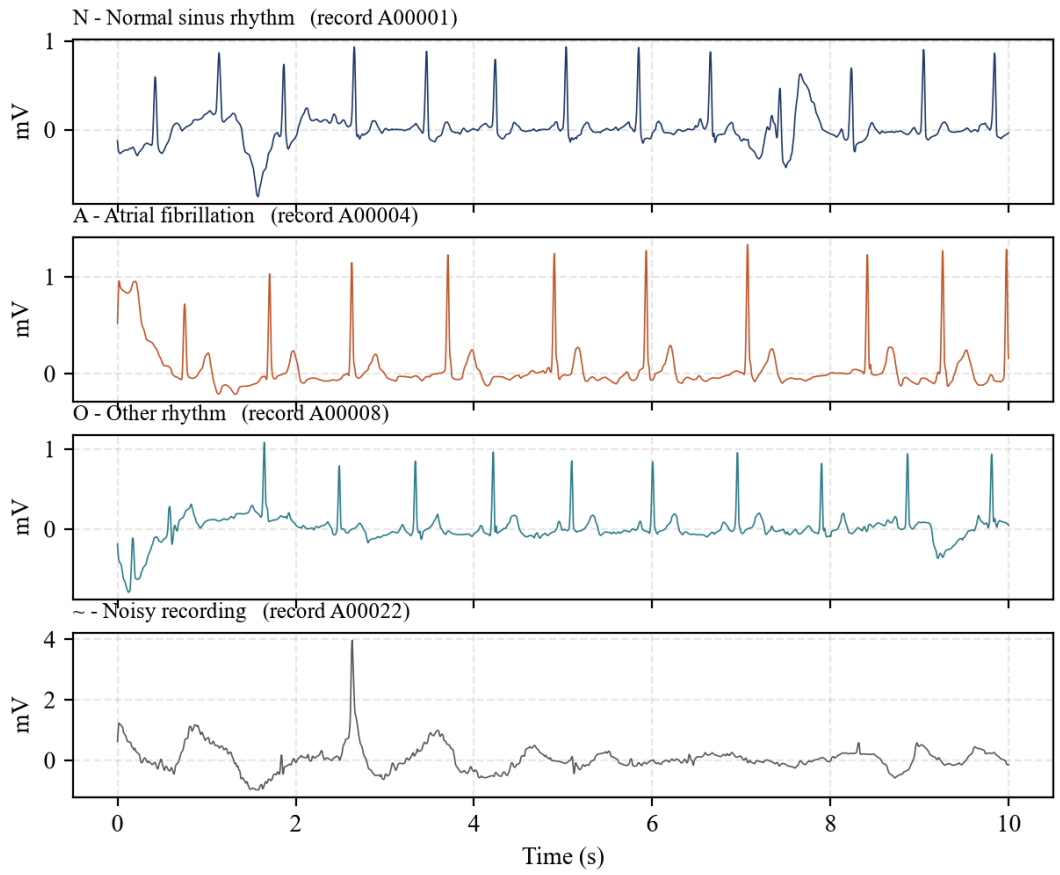


Fig 3.1 Ten seconds of each of the four rhythm classes of the PhysioNet/CinC 2017 dataset

3.2 CONVOLUTIONAL NEURAL NETWORKS

A convolutional layer slides a small learned filter along its input. For a one-dimensional input x, a filter w of length K and a bias b, the output at position n is given by Eq. 3.1. A two-dimensional layer does the same over both axes of an image. Early layers learn simple shapes

such as slopes and peaks; later layers combine them into patterns such as a whole QRS complex.

$$y[n] = b + \Sigma_k\, w[k] \cdot x[n+k], \quad k = 0 \ldots K-1 \tag{3.1}$$

Two refinements were used. A residual block adds its input to its output, which gives the gradient a direct path backwards and allows deeper networks to be trained [11]. A squeeze-and-excitation (SE) block [12] reduces each of the C channels of a feature map to its mean z, and rescales the channels by learned weights s (Eq. 3.2), where σ is the logistic sigmoid and W_1, W_2 are small dense layers.

$$s = \sigma(\, W_2 \cdot \mathrm{ReLU}(\, W_1 \cdot z \,)\,), \quad \hat{y}^c = s^c \cdot y^c \tag{3.2}$$

3.3 GRAPH NEURAL NETWORKS

A graph neural network attaches a learned vector h_v to every node v of a graph and updates it from messages sent along the edges. In one round of message passing, every node collects messages from its neighbours N(v) and combines them with its own state (Eq. 3.3), where M and U are small multi-layer perceptrons. Because the same weights are used at every node, one trained network can be applied to a graph of any size, which is exactly what a circuit of unknown size requires.

$$h_v^{(l+1)} = U(\, h_v^{(l)},\ \Sigma_{u \in N(v)}\, M(\, h_u^{(l)}, e_{uv} \,)\,) \tag{3.3}$$

3.4 STATIC TIMING ANALYSIS

Static timing analysis checks whether a digital circuit meets its clock period without simulating it [9]. The design is represented as a directed acyclic graph of pins. The arrival time (AT) of a pin is the latest time at which a signal can reach it; it is computed in topological order as the maximum, over all arcs into the pin, of the arrival time at the source of the arc plus the delay of the arc (Eq. 3.4). The slack of a timing endpoint is its required arrival time (RAT), set by the clock period, minus its arrival time (Eq. 3.5). A negative slack is a timing violation.

$$\mathrm{AT}(v) = \max_{u \to v} [\, \mathrm{AT}(u) + d(u, v) \,] \tag{3.4}$$

$$\mathrm{slack}(v) = \mathrm{RAT}(v) - \mathrm{AT}(v) \tag{3.5}$$

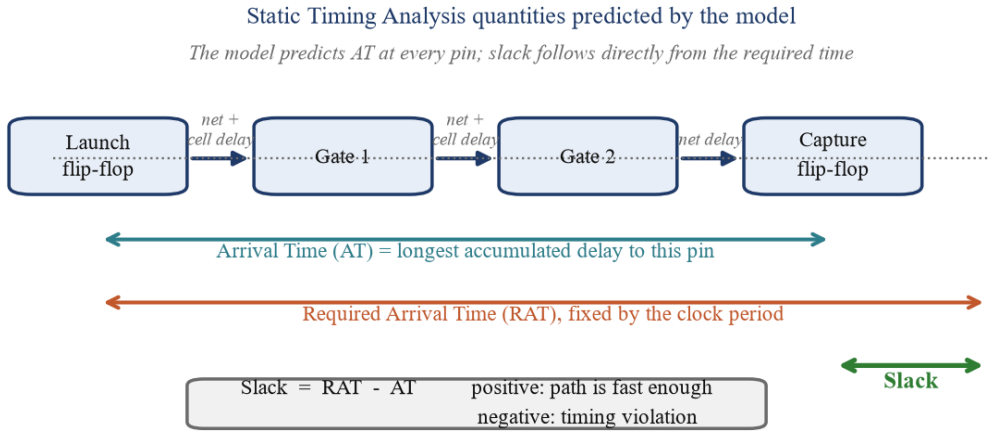


Fig 3.2 Arrival time, required arrival time and slack on a timing path

The delay of a gate is not computed from a formula but read from the characterisation data of the standard-cell library, stored in the Liberty format as a table indexed by the input slew and the output capacitive load. Wire delay depends on the length and capacitance of the wire, which are known accurately only after routing; this is why timing before routing is difficult to estimate and why a learned predictor is useful (Fig 3.3).

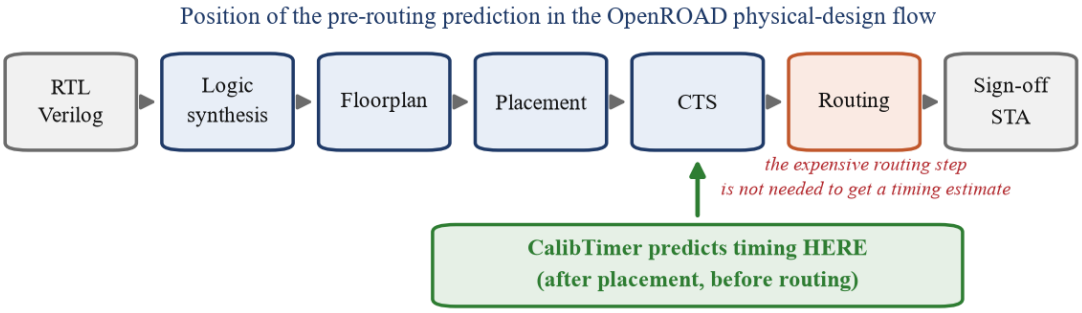


Fig 3.3 Position of the pre-routing prediction in the OpenROAD flow

3.5 PAPERS STUDIED

The papers studied are summarised in Table 3.1. The figures quoted in the table were checked against the papers themselves.

Guo et al. [1] was the starting paper for the timing work. It represents a placed design as a heterogeneous graph of pins with separate net and cell edges, embeds the nets with three rounds of message passing, and then propagates arrival time level by level as a timing engine

does, reading cell delays through a learned interpolation of the Liberty tables. It publishes a benchmark of 21 sky130 designs divided into 14 for training and 7 for testing, and reports an arrival-time R^2 averaged over the test designs of 0.8957. CalibTimer follows its structure and uses the same test designs.

Bodhe et al. [2] (E2ESlack) build an end-to-end framework around that model, add a parser for the timing files and an algorithmic computation of the required arrival time, and train on 23 circuits. They report arrival-time R^2 values for the same seven test designs whose mean is 0.9169. Barboza et al. [6] predict timing before routing with feature-based machine learning and showed that the pessimism of classical estimates can be reduced. Chang and Kim [7] and Li et al. [8] apply graph neural networks, the latter with graph attention, to the same pre-routing problem. Zhang et al. [5] study how a timing predictor can be moved between technology nodes, and Hu et al. [4] model RC interconnect timing with a physics-informed tree network. Bairamkulov and Friedman [3] give the graph formulation of timing and clocking that underlies all of them.

For the ECG work, the PhysioNet/CinC 2017 challenge paper by Clifford et al. [10] defines the dataset and its four classes, and the residual [11] and squeeze-and-excitation [12] papers define the blocks used in Phases 1 and 2. Lin et al. [13] introduce the focal loss used to cope with the class imbalance.

Table 3.1 Summary of the papers studied

Reference	Task	Method	Data	Reported result
Guo et al. [1], DAC 2022	Pre-routing AT and slack	Timing-engine-inspired GNN, learned LUT read	21 sky130 designs, 14/7 split	Test AT R^2 avg. 0.8957
Bodhe et al. [2], E2ESlack	Pre-routing slack, TNS/WNS	Model of [1] + parser + RAT algorithm	34 circuits, 23 for training	Same 7 test designs, AT R^2 mean 0.9169
Barboza et al. [6], DAC 2019	Pre-routing timing	Feature-based ML	Placed designs	Reduced pessimism vs. classical estimates
Chang and Kim [7], 2024	Pre-route timing and optimisation	GNN models	Benchmark designs	GNN-based timing prediction
Li et al. [8], 2025	Pre-routing slack	Graph attention network	Benchmark designs	Scalable slack prediction
Zhang et al. [5], TCAD 2025	Cross-node pre-routing timing	Transfer across technology nodes	Multiple nodes	Prediction across nodes
Hu et al. [4], NeurIPS 2023	RC interconnect timing	Physics-informed tree GNN	RC trees	Delay and slew vs. SPICE
Clifford et al. [10], 2017	AF detection from short ECG	Challenge and dataset	8,528 recordings	Four-class benchmark

3.6 GAP IDENTIFIED

Two points came out of the literature study. First, published pre-routing predictors report a single number, the predicted arrival time or slack, with no statement of how far it can be trusted on a new design; a designer who must decide whether a path is safe needs an interval, not just a point. Second, comparisons between papers are difficult because they differ in how the coefficient of determination is computed and over which pins. The work was therefore directed at a model that follows the established timing-engine structure, is evaluated per design on the official unseen designs over the pins that carry real timing labels, and attaches a calibrated interval to its slack predictions.

CHAPTER 4: TOOLS AND ENVIRONMENT

4.1 SOFTWARE ENVIRONMENT

All software used is open source; the versions in Table 4.1 are those installed in the project environments. No commercial licence was needed at any stage.

Table 4.1 Software used

Item	Version	Used for
Python	3.11.7	All data processing, training and analysis
PyTorch (CUDA 12.1)	2.5.1	Graph neural network of Phase 3
PyTorch (CUDA 12.6)	2.12.1	Two-dimensional networks of Phase 2
TensorFlow / Keras	2.21.0	One-dimensional networks of Phase 1
NumPy, SciPy, pandas	2.4.6, 1.17.1, 3.0.5	Arrays, signal processing, tables
PyWavelets, OpenCV	—	Wavelet transforms and image generation
scikit-learn	—	Splits, classification reports, confusion matrices
Matplotlib, NetworkX	3.11.2, 3.6.1	All figures, graph drawing
OpenROAD Flow Scripts	—	Synthesis, floorplan, placement and CTS
OpenSTA	3.1.0	Reference static timing analysis (training labels)
SkyWater sky130 PDK	sky130_fd_sc_hd	Standard cells, LEF and Liberty data
Windows Subsystem for Linux	Ubuntu	Linux environment for the design flow

4.2 HARDWARE

Table 4.2 Hardware used

Item	Detail
Processor platform	Personal laptop, Windows 11
Graphics processor	NVIDIA GeForce RTX 3050 Laptop GPU, 6 GB
Use	Training and inference of every model reported here

4.3 THE OPENROAD FLOW AND THE SKY130 LIBRARY

Each benchmark design was synthesised, floorplanned, placed and given a clock tree with the OpenROAD Flow Scripts on the sky130 high-density library, and its reference timing was produced with OpenSTA. For each design the flow leaves the files listed in Table 7.1, which are the input of the parser described in Chapter 7. The Liberty library contains 1,241 timing arcs, each with seven-by-seven delay and slew tables.

4.4 THE BENCHMARK DESIGNS

Twenty-seven designs were used. Table 4.3 gives their size, read directly from the DEF and SDC files: the die dimensions, the number of placed components and of logic cells (components other than fillers, taps and decoupling cells), the number of nets and input-output pins, and the clock period. The designs span almost four orders of magnitude (Fig 4.1), from 24 logic cells in priority_enc to 90,108 in aes256.

Table 4.3 The 27 benchmark designs (read from their DEF and SDC files)

Design	Group	Die (µm)	Comp.	Logic cells	Nets	I/O	Clock (ns)
BM64	train	710 × 721	57,983	10,809	10,615	1030	20
aes128	train	1355 × 1365	238,867	66,635	62,048	387	11.43
aes192	test	1647 × 1658	327,794	72,884	68,861	451	12.37
aes256	train	1533 × 1544	309,677	90,108	83,541	515	15
aes_cipher	train	703 × 713	65,607	17,445	17,106	393	10.09
alu4bit	train	71 × 71	194	142	168	21	10
blabla	train	1056 × 1067	123,336	17,359	15,715	1422	65
cic_decimator	train	142 × 153	2,676	877	899	44	10
cla8b	calib.	40 × 40	171	62	88	29	none
des	train	683 × 694	59,687	16,526	16,063	188	20
gcd	calib.	80 × 80	751	484	517	56	1.1
genericfir	train	549 × 559	34,719	9,975	9,982	61	10
jpeg_encoder	test	1684 × 1695	337,006	65,813	61,479	51	15.78
picorv32a	train	660 × 670	57,366	16,446	15,629	411	24.73
priority_enc	calib.	60 × 60	217	24	32	14	none
salsa20	train	1040 × 1051	121,934	21,837	20,749	79	36.73
seq_detector	calib.	36 × 36	131	25	27	6	10
spm	test	93 × 104	1,028	321	356	38	10
synth_ram	test	458 × 469	24,305	6,956	6,886	94	50
uni_counter4b	calib.	42 × 42	207	55	67	16	10
usb	train	169 × 180	3,376	934	955	46	12.55
usb_cdc_core	train	223 × 234	6,729	2,182	2,206	55	15.6
usbf_device	test	652 × 663	59,757	20,167	20,104	181	14.6
wbqspiflash	train	262 × 272	9,017	2,834	2,874	106	18.86
xtea	test	236 × 247	8,136	2,892	3,062	262	26.03
y_huff	test	565 × 576	42,451	14,506	14,527	749	11.35
zipdiv	train	176 × 186	4,022	1,236	1,296	109	19.09

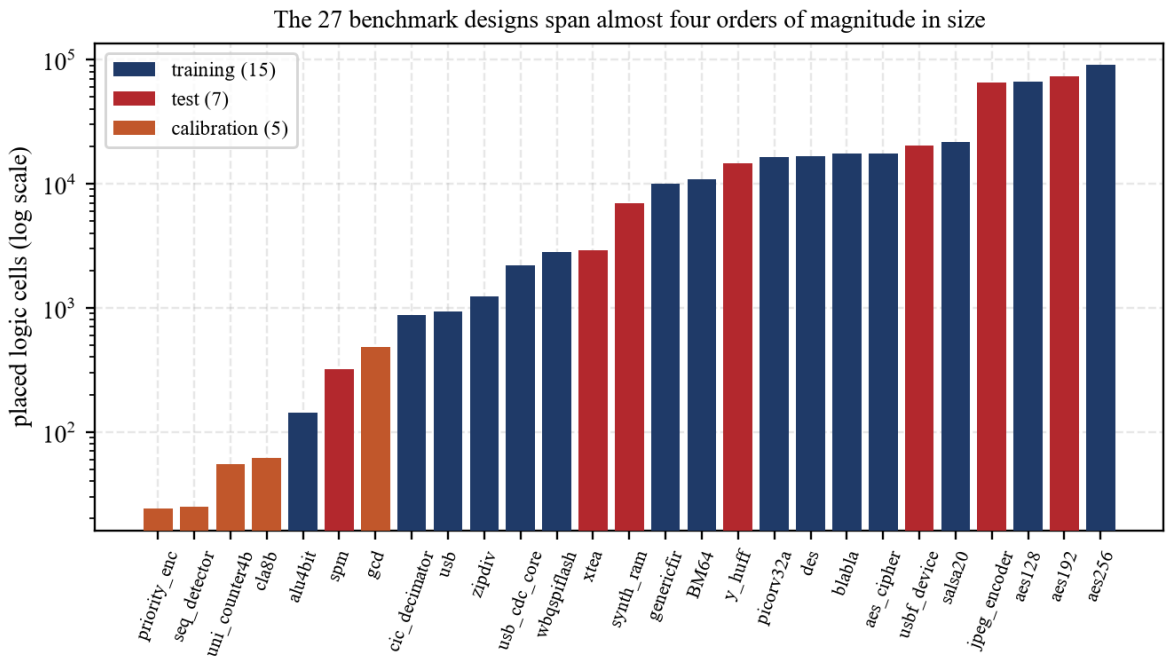


Fig 4.1 Size of the 27 benchmark designs (logic cells, log scale)

Fig 4.2 shows two of the test designs exactly as they were placed, drawn from the coordinates in their DEF files and the cell sizes in the library LEF. The small serial-parallel multiplier spm has 321 logic cells, 64 of them flip-flops; the USB function controller usbf_device has 20,150.

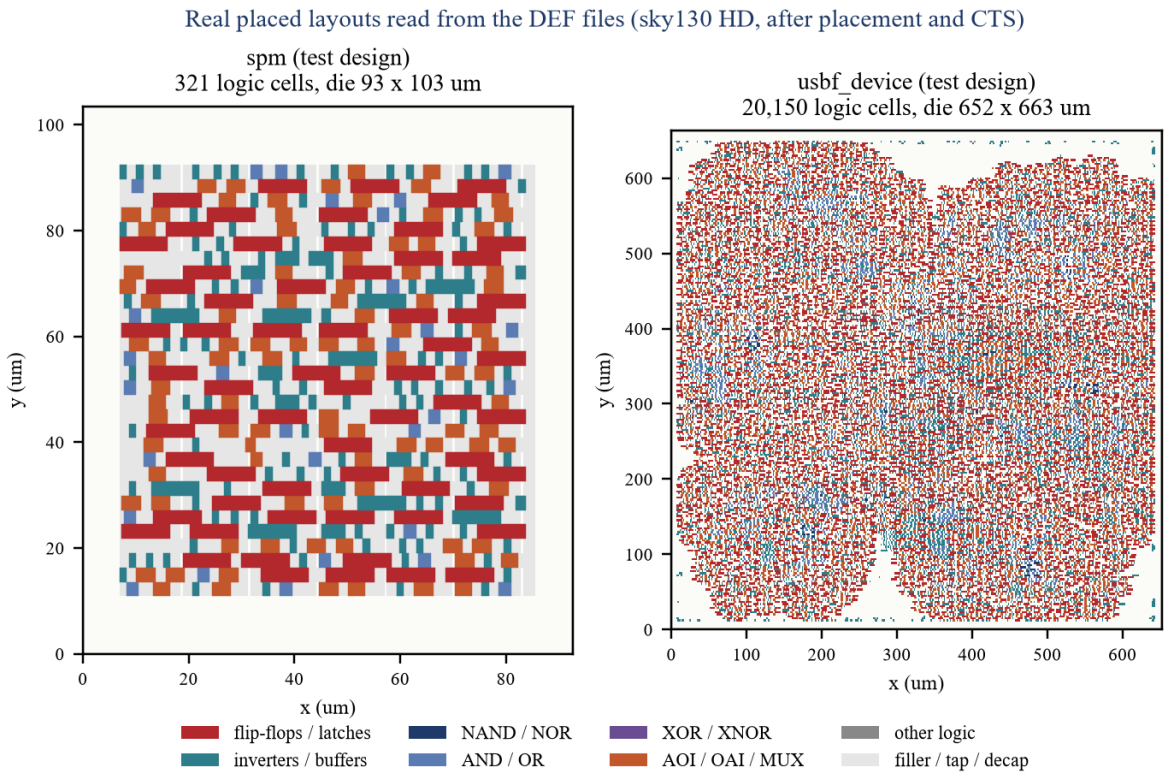


Fig 4.2 Real placed layouts of two test designs, drawn from their DEF files

CHAPTER 5: PHASE 1: ONE-DIMENSIONAL CNN ON ECG

5.1 PROBLEM STATEMENT

The task of the first phase was to classify short single-lead ECG recordings into four classes: normal sinus rhythm, atrial fibrillation, other rhythm, and recordings too noisy to classify. It is a one-dimensional pattern-recognition problem on a sampled signal, which is the natural setting for a one-dimensional convolutional network: its filters behave like matched filters tuned to the shape of a heartbeat.

5.2 THE DATASET

The PhysioNet/CinC Challenge 2017 dataset [10] contains 8,528 single-lead recordings sampled at 300 Hz, from 9 to 61 seconds long, each with one label. The classes are strongly unbalanced (Table 5.1, Fig 5.1): a classifier that always answers “normal” would already be 59.5 per cent accurate, which is why the macro F1 score is used to compare the models.

Table 5.1 Classes of the PhysioNet/CinC 2017 dataset

Label	Class	Recordings	Share
N	Normal sinus rhythm	5,076	59.5 %
A	Atrial fibrillation	758	8.9 %
O	Other rhythm	2,415	28.3 %
~	Too noisy to classify	279	3.3 %
	Total	8,528	100 %

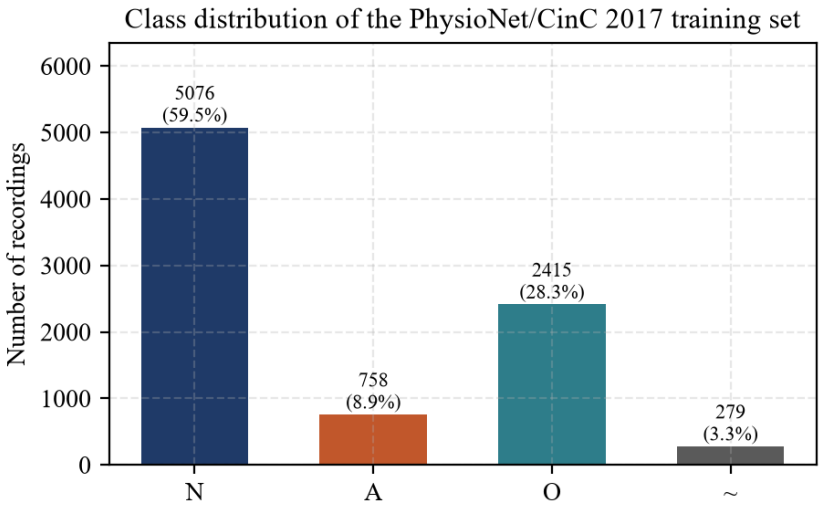


Fig 5.1 Class distribution of the dataset

5.3 PREPROCESSING

Every recording was brought to a fixed length of 9,000 samples, thirty seconds at 300 Hz: longer records were truncated and shorter ones extended by edge padding, which repeats the last sample rather than inserting an abrupt step of zeros. Each record was normalised to zero mean and unit variance using its own statistics, so that differences in electrode gain do not reach the network. The data were divided into training, validation and test subsets by recording, in a stratified manner.

5.4 NETWORK ARCHITECTURE

Several networks were compared, from a plain convolutional baseline to a residual network with squeeze-and-excitation blocks. The best is shown in Fig 5.2: a stem convolution of 128 filters of length 25, four residual blocks of the same width, each with an SE gate that narrows the 128 channels to 16 and back, global average pooling, and two dense layers with dropout. A filter of 25 samples spans about 83 ms, the order of a QRS complex, so one filter can respond to a whole beat feature.

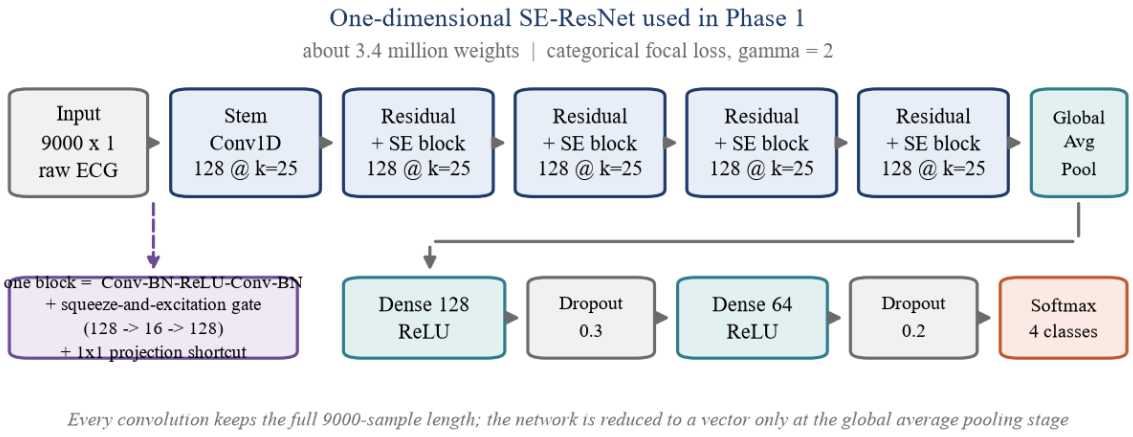


Fig 5.2 Architecture of the one-dimensional SE-ResNet

5.5 TRAINING

Because the normal class is eighteen times as frequent as the noisy class, the focal loss [13] was used (Eq. 5.1). It scales the cross-entropy of each sample by $(1 - p_t)^{\gamma}$, where p_t is the probability given to the true class, so the gradient is dominated by the samples still classified wrongly; $\gamma = 2$ was used.

$$FL(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t) \quad (5.1)$$

Table 5.2 Training settings of the one-dimensional networks

Setting	Value
Input	9,000 samples × 1 channel
Loss	Categorical focal loss, $\gamma = 2$
Optimiser	Adam, learning rate 1e-3
Batch size	32
Early stopping	on validation macro F1
Learning-rate schedule	reduce on plateau
Test set	1,706 held-out recordings

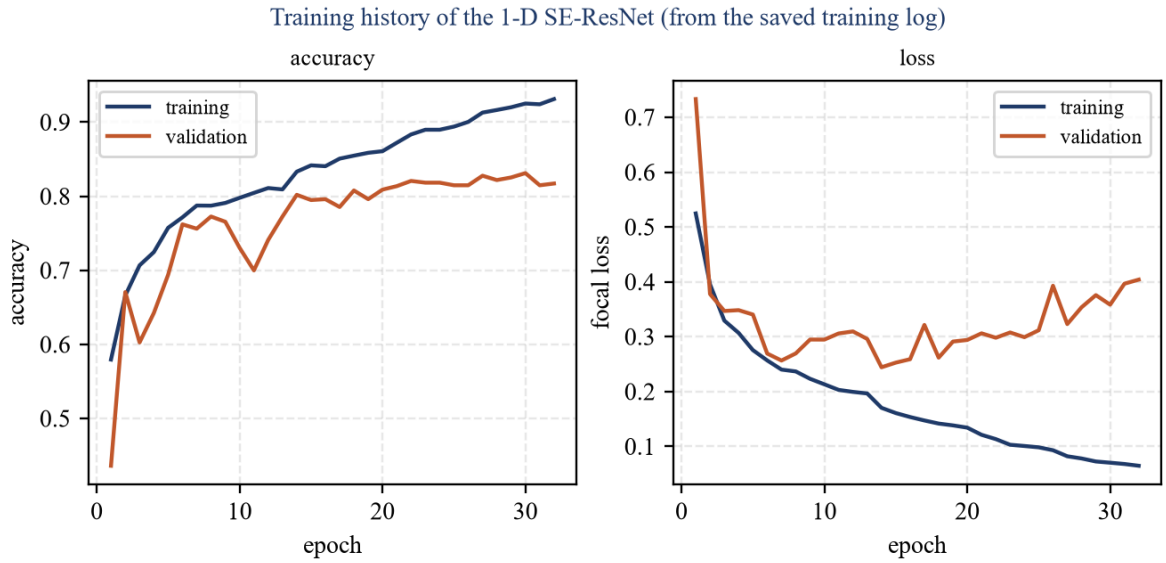


Fig 5.3 Training history of the one-dimensional SE-ResNet

The training history (Fig 5.3) was read from the log saved during training. The best validation accuracy, 0.831, was reached at epoch 30, and the model of that epoch was kept.

5.6 RESULTS OF PHASE 1

Table 5.3 Results of the one-dimensional networks on the test set

Model	Test accuracy	Macro F1	Test loss
Baseline 1-D CNN	0.7790	0.7169	0.5828
1-D CNN with four convolution blocks	0.7315	0.6840	0.8184
Simple CNN with squeeze-and-excitation	0.7761	0.6942	0.6312
SE-ResNet, 128 filters, kernel 25	0.8048	0.7302	0.2402

Table 5.4 Per-class results of the one-dimensional SE-ResNet on the test set

Class	Precision	Recall	F1 score	Recordings
N normal	0.8936	0.8601	0.8765	1,015
A atrial fibrillation	0.8258	0.7171	0.7676	152
O other rhythm	0.7125	0.7081	0.7103	483
~ noisy	0.4188	0.8750	0.5665	56
Macro average	0.7127	0.7901	0.7302	1,706
Weighted average	0.8207	0.8048	0.8096	1,706

The SE-ResNet reached 80.5 per cent accuracy and 0.730 macro F1 on 1,706 held-out recordings, with a per-class recall of 0.86 for normal rhythm, 0.72 for atrial fibrillation, 0.71 for other rhythms and 0.88 for noisy records. The weakest class is “other rhythm”, which is a collection of everything that is neither normal nor atrial fibrillation and so has no single waveform signature. Simply adding convolution blocks made the plain network worse (0.684 against 0.717); what helped was the kind of depth, residual blocks with attention, which also cut the test loss from 0.583 to 0.240.

CHAPTER 6: PHASE 2: TWO-DIMENSIONAL CNN ON TIME-FREQUENCY IMAGES

6.1 MOTIVATION

A common idea in the literature is to convert a one-dimensional signal into a time-frequency image and classify it with an image network, which makes the frequency content explicit and opens up the large body of two-dimensional architectures. Phase 2 tested whether that idea holds for this dataset (Fig 6.1).

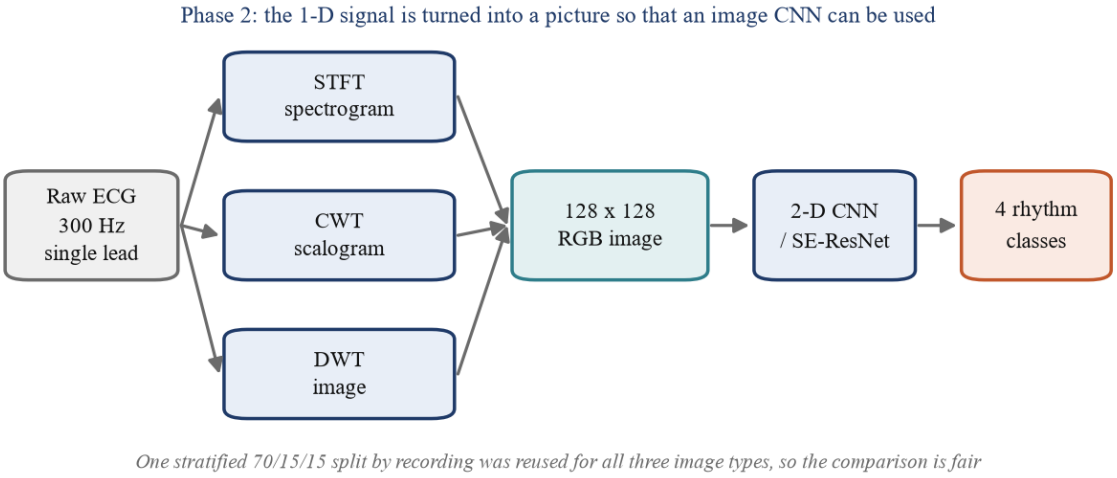


Fig 6.1 From a one-dimensional signal to a two-dimensional image

6.2 GENERATING THE IMAGES

Three representations were generated for all 8,528 recordings as 128 × 128 colour images (Table 6.1, Fig 6.2). One stratified split into 5,968 training, 1,280 validation and 1,280 test recordings was made once and reused for every image type, so that the networks differ only in the representation they see.

Table 6.1 The three time-frequency representations

Representation	Transform	What it shows
Spectrogram	Short-time Fourier transform, 64-sample window, 32 overlap	Energy in fixed frequency bands against time
Scalogram	Continuous wavelet transform	Energy against scale and time; finer time resolution at high frequency
DWT image	Discrete wavelet transform, Daubechies-4, multi-level	Decomposition levels stacked as rows; compact but coarse

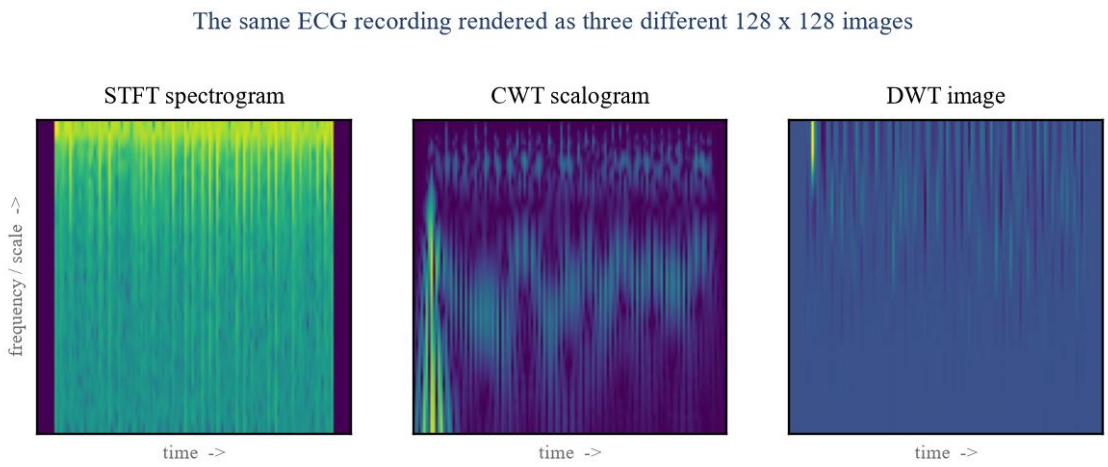


Fig 6.2 One recording as a spectrogram, a scalogram and a DWT image

6.3 ARCHITECTURE SWEEP AND KERNEL ABLATION

A configurable network was swept over two, three and four convolution blocks and one or two dense layers, giving six architectures, each trained on each of the three image types: eighteen networks in all (Fig 6.3). The DWT image was the worst representation in all six configurations, while the spectrogram and the scalogram were close, the spectrogram ahead in four configurations and the scalogram in two. Deeper networks were better in all but one case, and the best of the eighteen, four blocks with one dense layer on scalograms, reached 0.641 macro F1. A further ablation compared kernel sizes on the same task (Table 6.2).

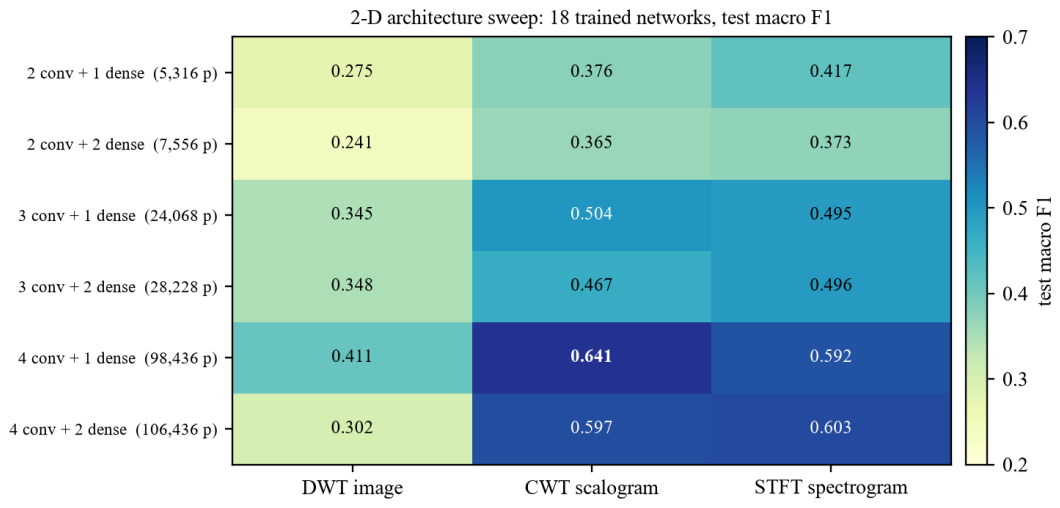


Fig 6.3 Test macro F1 of the 18 networks of the architecture sweep

Table 6.2 Kernel-size ablation (simple two-dimensional CNN)

Kernel	Test accuracy	Macro F1
2 × 2	0.6195	0.3003
3 × 3	0.6438	0.3973
mixed	0.6219	0.3171

6.4 RESIDUAL NETWORKS

Two deliberate translations of the Phase 1 network were then built so that the only remaining variable was signal against image (Fig 6.4). The squeeze-and-excitation version keeps the residual blocks and attention gates and uses 3 × 3 kernels with widths of 64, 128 and 128; the plain residual baseline uses 5 × 5 kernels, the same kernel area as the one-dimensional kernel of 25. Both were trained with the focal loss and with random-crop-and-repeat-pad augmentation.

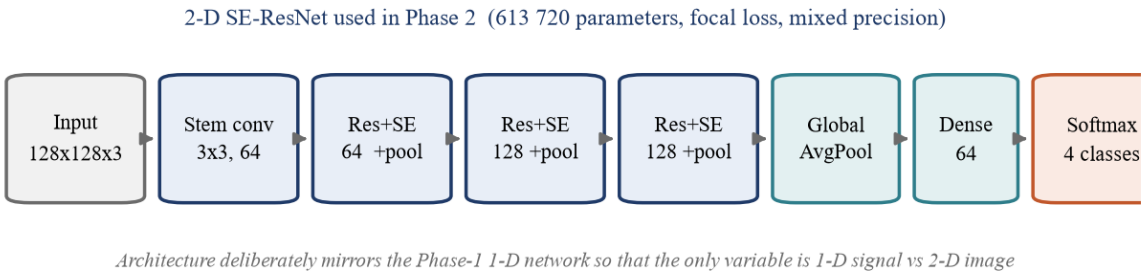


Fig 6.4 Architecture of the two-dimensional SE-ResNet

6.5 RESULTS OF PHASE 2

Table 6.3 Best results of the two-dimensional networks on the test set

Model and input	Parameters	Test accuracy	Macro F1
2-D SE-ResNet, spectrograms	613,720	0.7586	0.6605
2-D ResNet baseline, spectrograms	—	0.7539	0.6536
Sweep best: 4 conv + 1 dense, scalograms	98,436	0.7227	0.6408
Sweep best on spectrograms (4 conv + 2 dense)	106,436	0.7000	0.6027
Sweep best on DWT images (4 conv + 1 dense)	98,436	0.5312	0.4109

Table 6.4 Per-class results of the two-dimensional SE-ResNet on the test set

Class	Precision	Recall	F1 score	Recordings
N normal	0.7878	0.9357	0.8554	762
A atrial fibrillation	0.7706	0.7434	0.7568	113
O other rhythm	0.7277	0.4050	0.5204	363
~ noisy	0.4219	0.6429	0.5094	42
Macro average	0.6770	0.6817	0.6605	1,280

The weakness of the image route is concentrated in the “other rhythm” class, whose recall falls to 0.405 against 0.708 for the one-dimensional network: the fine timing irregularities that separate it from normal rhythm are blurred when the recording is reduced to a 128 × 128 image.

The residual baseline also overfitted clearly: 0.813 macro F1 on the training data against 0.654 on the test data. Turning a signal of 9,000 values into an image of 49,152 values adds input without adding information, so with a fixed number of recordings the network has more room to memorise.

6.6 WHY THE APPROACH WAS DISCONTINUED

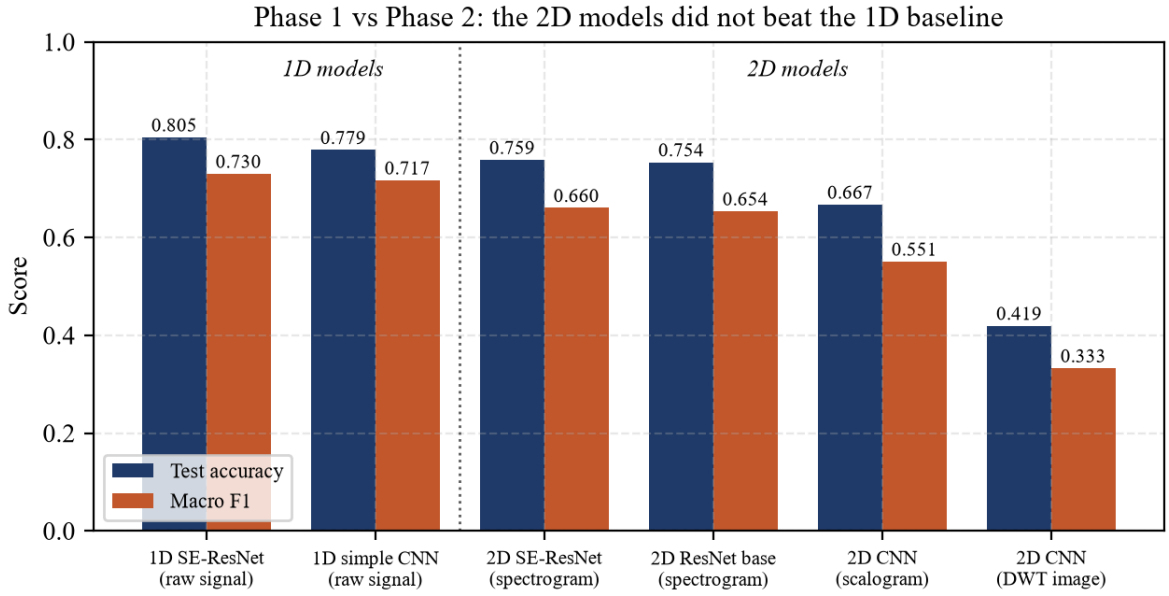


Fig 6.5 Accuracy and macro F1 of the one- and two-dimensional models

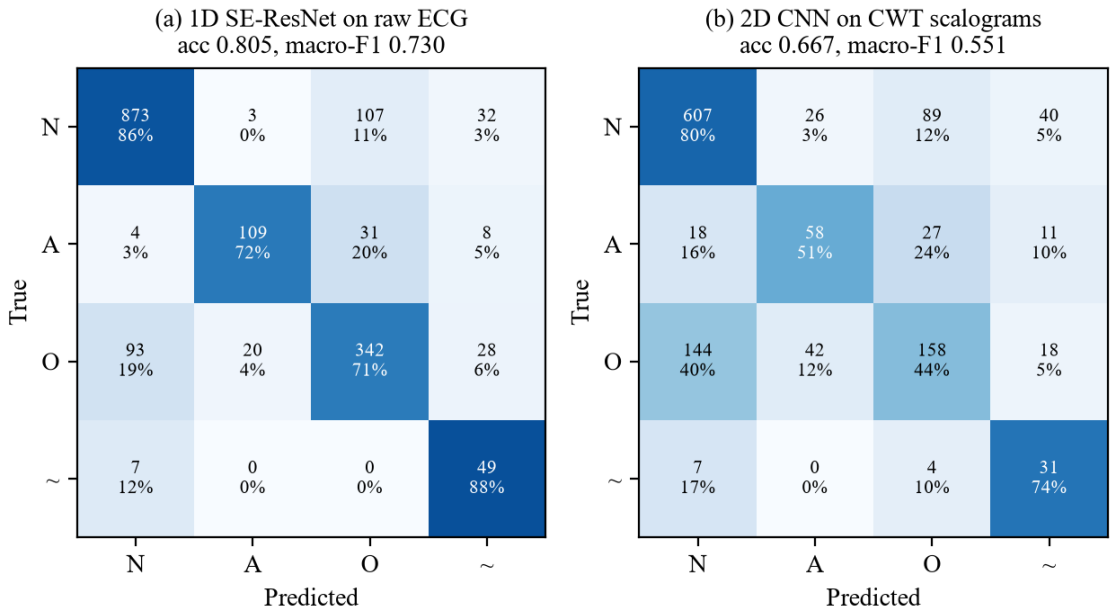


Fig 6.6 Confusion matrices: 1-D SE-ResNet and a 2-D CNN on scalograms

Table 6.5 Best model of each phase

	Phase 1: 1-D, raw signal	**Phase 2: 2-D, image**
Test accuracy	0.8048	0.7586
Macro F1	0.7302	0.6605
Input values per recording	9,000	49,152
Preprocessing	normalisation only	a transform of every record

The conclusion was negative. On every measure the best two-dimensional network was below the one-dimensional network it had been translated from, while costing a transform of every recording and a much larger input. A one-dimensional convolution can already learn frequency-selective filters, and at this dataset size the extra input dimension hurt more than the explicit frequency axis helped. The result was reviewed with the supervisor, and rather than keep tuning an idea that had been given a fair test and lost, the internship was re-scoped to a problem whose natural structure is a graph.

CHAPTER 7: PHASE 3: CALIBTIMER — DATA AND GRAPH CONSTRUCTION

7.1 OVERVIEW

CalibTimer takes a design that has been placed but not routed and predicts, for every pin, the arrival time and slew, and the delay of every wire and every gate arc; slack follows from the predicted arrival time. The system has three parts: a parser that turns the design files into a graph, the network, and a calibration step that turns a predicted slack into an interval. This chapter covers the first part.

7.2 DESIGN FILES AND PARSER

Table 7.1 Files read for each design

File	Content	Used for
.synthesis_preroute.v	Structural Verilog netlist	Pins and their connections
.def	Placement (DEF)	x, y of every cell and pin; die area
.spef	Extracted parasitics (SPEF)	Capacitance of every net
.sdf	Annotated delays (SDF)	Cell-arc and net delay labels
.at.sdf	Timing result of OpenSTA	True arrival time, slew and RAT per pin
.cts_1.sdc	Timing constraints (SDC)	Clock period and input delays
sky130 .lib	Liberty library	7 × 7 delay and slew tables per arc

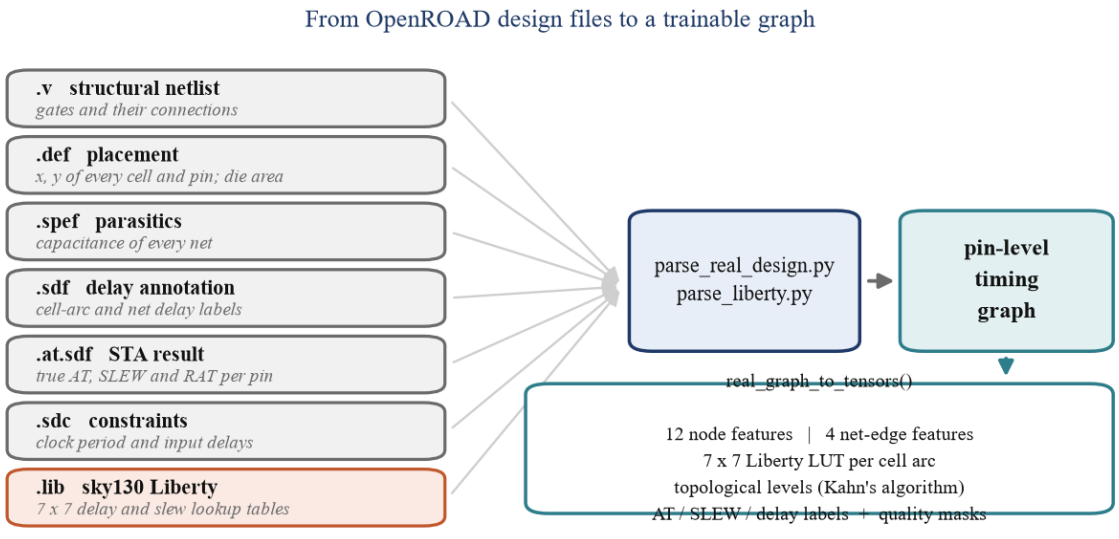


Fig 7.1 From the design files to a trainable graph

Writing the parsers was a substantial part of the phase, because the formats are verbose and differ in dialect. Two defects found and fixed during the work were in this stage: timing endpoints were at first identified wrongly, and the true arrival times read from the .at.sdf file were at first parsed and then silently replaced by a propagated estimate. A third, found only by the tests of Chapter 9, is described in Section 9.4. Results obtained before each fix are not reported.

7.3 GRAPH CONSTRUCTION

Each design becomes a directed graph in which a node is one pin of a cell or one primary input or output (Fig 7.2). A net edge joins the pin that drives a net to every pin the net feeds and carries the wire delay; a cell edge joins an input pin of a gate to its output and is one timing arc of the gate.

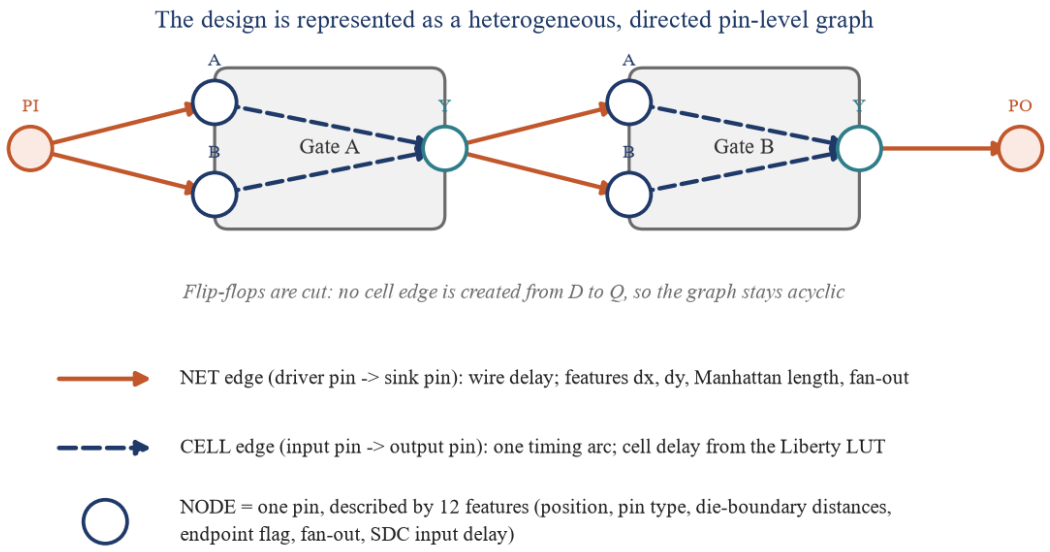


Fig 7.2 The heterogeneous pin-level timing graph (schematic)

Flip-flops need care. A sequential cell is recognised by its clock pin, and no cell edge is created from its data input to its output, since the output does not depend on the input within one clock cycle. Without this rule a flip-flop whose output feeds back into its own input cone creates a loop, the graph stops being acyclic and no topological order exists.

```
SEQUENTIAL_CLOCK_PORTS = {"CLK", "CLK_N", "GCLK"}
is_sequential = any(p in SEQUENTIAL_CLOCK_PORTS for p in conns.keys())
...
if is_sequential:
```

```
    continue          # pins become nodes, but no D -> Q cell edge
for ik in in_keys:
    cell_edges.append((ik, out_key, cell_type))
```

Fig 7.3 shows the actual graph that the parser builds for the smallest calibration design, seq_detector, drawn by topological level: 64 pins, 43 net edges and 20 cell edges over 8 levels. Every one of the 27 designs was checked in the same way (Section 9.8); the graph of the largest, jpeg_encoder, has 1,485,105 pins.

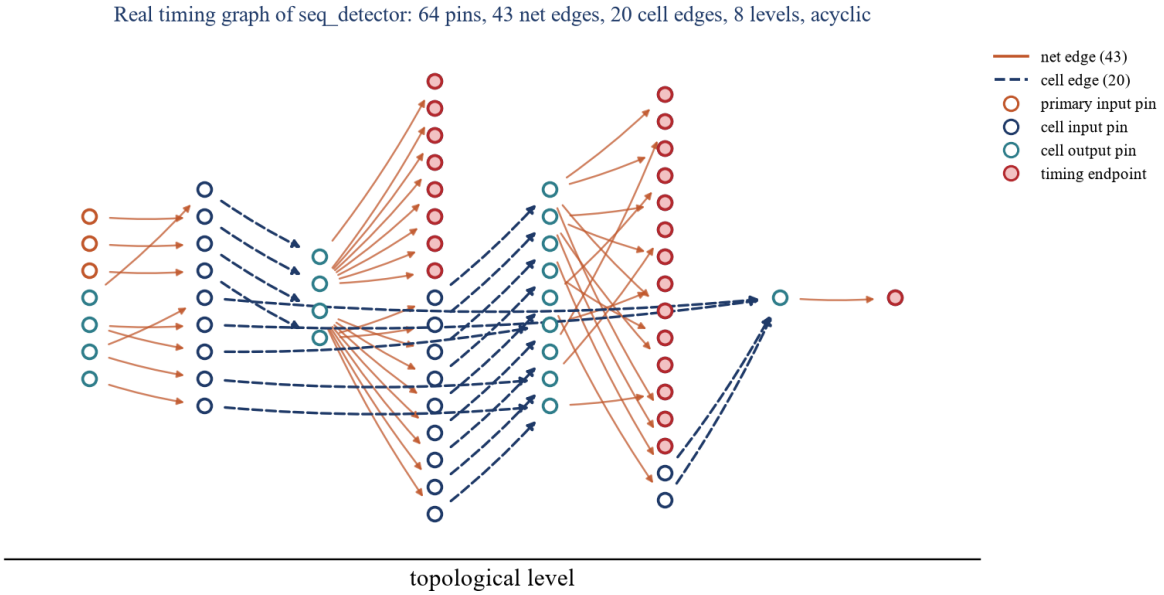


Fig 7.3 The real timing graph of seq_detector, built by the parser

7.4 NODE AND EDGE FEATURES

Every pin is described by twelve numbers, all available before routing (Table 7.2). Every net edge carries four: the horizontal and vertical separation of its pins, their Manhattan distance and the fan-out of the driver. Every cell edge carries a one-hot code of the cell type and the seven-by-seven delay and slew tables of its arc, read from the Liberty file.

Table 7.2 The twelve node features

Index	Feature	Reason
0, 1	x and y position, normalised	Wire delay depends on distance
2, 3, 4	Primary input, cell input, cell output	Pin types behave differently
5–8	Distance to the four die edges	Wiring environment near the boundary
9	Timing endpoint flag	Pins at which slack is evaluated
10	Fan-out, normalised	Load seen by a driver
11	Input delay from the SDC, normalised	Arrival time at a primary input (see Section 9.4)

Fig 7.4 shows one of the 1,241 Liberty arcs exactly as the model receives it: the delay of a two-input NAND gate grows with output load and with input slew, from 0.015 ns to 1.378 ns across the table.

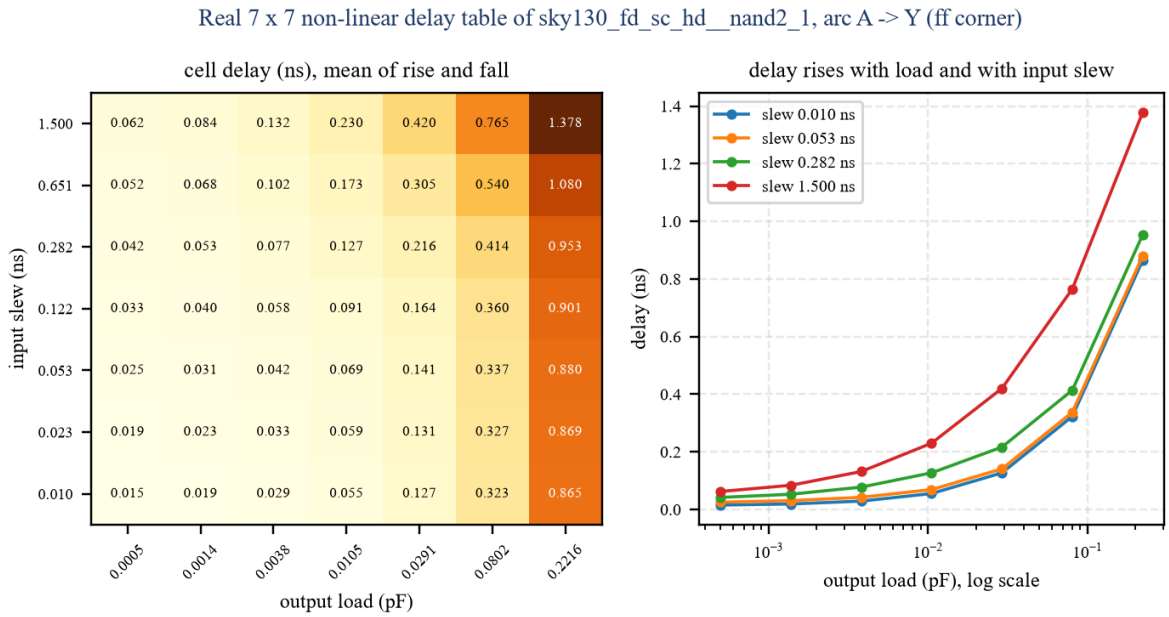


Fig 7.4 A real Liberty delay table (sky130_fd_sc_hd__nand2_1, A to Y)

7.5 DIVISION OF THE DESIGNS

The 27 designs are divided into three groups that never overlap (Fig 7.5, Table 7.3). Seven are the test designs of the reference work [1] and are never trained on; five small designs were generated separately and are reserved for calibration, since training on them would destroy the statistical validity of the calibration; the remaining fifteen are used for training. The separation is enforced in code: the training script holds the test and calibration names in fixed sets and stops if either appears among the training designs. This guard was added after exactly such an overlap had been found in an earlier version.

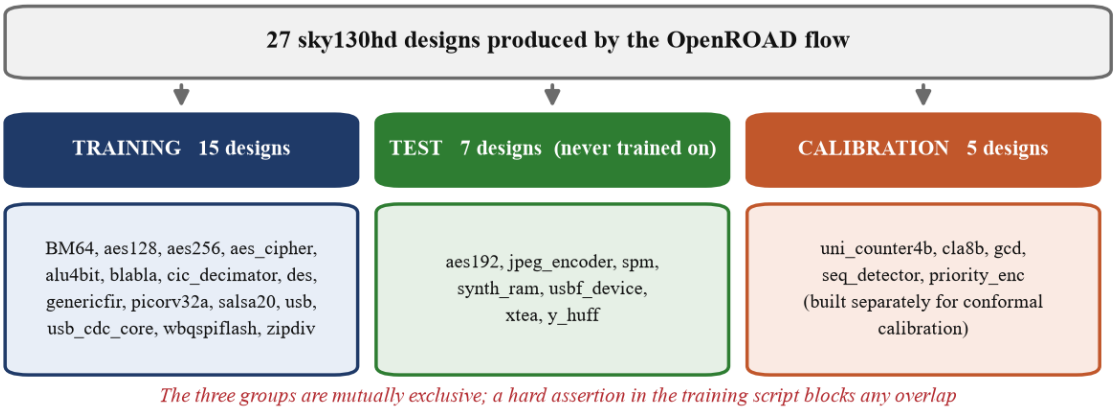


Fig 7.5 Division of the 27 designs into training, test and calibration groups

Table 7.3 The three groups of designs

Group	Count	Designs
Training	15	BM64, aes128, aes256, aes_cipher, alu4bit, blabla, cic_decimator, des, genericfir, picorv32a, salsa20, usb, usb_cdc_core, wbqspiflash, zipdiv
Test	7	aes192, jpeg_encoder, spm, synth_ram, usbf_device, xtea, y_huff
Calibration	5	uni_counter4b, cla8b, gcd, seq_detector, priority_enc

```
PAPER_TEST_DESIGNS  = {"jpeg_encoder", "usbf_device", "aes192", "xtea",
                       "spm", "y_huff", "synth_ram"}
CALIBRATION_DESIGNS = {"uni_counter4b", "cla8b", "gcd",
                       "seq_detector", "priority_enc"}
train = [d for d in all_designs if d not in PAPER_TEST_DESIGNS
                                and d not in CALIBRATION_DESIGNS]
assert not [d for d in train if d in PAPER_TEST_DESIGNS],  "LEAKAGE GUARD"
assert not [d for d in train if d in CALIBRATION_DESIGNS], "LEAKAGE GUARD"
```

CHAPTER 8: PHASE 3: CALIBTIMER — MODEL AND TRAINING

8.1 NETWORK ARCHITECTURE

The network has two stages (Fig 8.1). The first embeds every pin; the second propagates arrival time and slew through the graph in topological order, exactly as a timing engine does. The whole network has 40,730 trainable parameters.

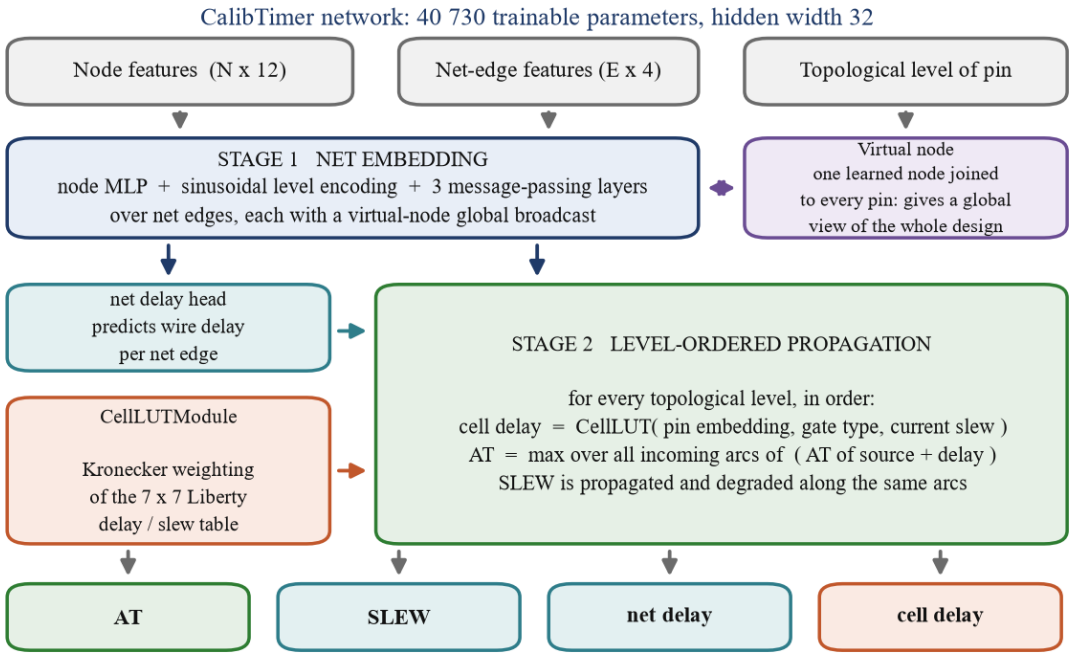


Fig 8.1 Block diagram of the CalibTimer network

In the first stage the twelve node features pass through a small multi-layer perceptron, and a sinusoidal encoding of the pin's topological level is added. Three rounds of message passing (Eq. 3.3) follow along the net edges, each with a residual update. A single virtual node, joined to every pin, is updated from the mean of all pin embeddings and broadcast back before each round, which gives every pin a path of length two to information from anywhere in the design.

8.2 READING THE LIBERTY TABLES

Gate delay is not predicted freely: it is read from the Liberty table T of the arc (Fig 8.2). From the pin embedding, the cell type and the current input slew, a small network predicts

seven weights α along the slew axis and seven weights β along the load axis, each normalised by a softmax. Their outer product weights the 49 entries of the table, and a small learned correction c is added (Eq. 8.1). The network therefore learns where to read the characterisation data rather than inventing delays, which is the main reason it needs only about forty thousand parameters.

$$\hat{d} = \Sigma_i \Sigma_j \alpha_i \beta_j T_{ij} + 0.2\, c \tag{8.1}$$

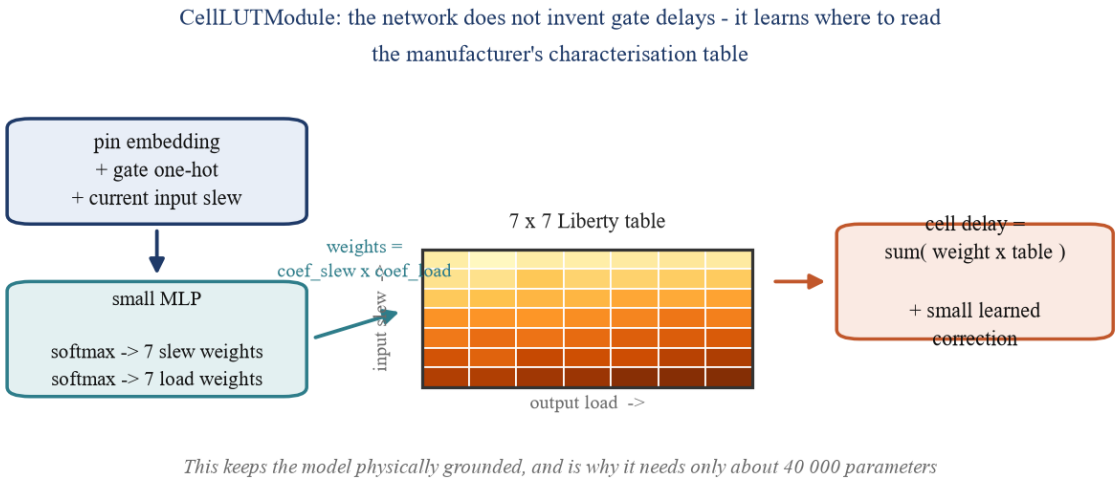


Fig 8.2 The CellLUTModule: a learned read of the Liberty table

```
coefs     = self.coef_mlp(feat)                       # (E, 2k)
coef_slew = torch.softmax(coefs[:, :self.k], dim=-1)  # alpha, slew axis
coef_load = torch.softmax(coefs[:, self.k:], dim=-1)  # beta, load axis
weight    = torch.einsum('ei,ej->eij', coef_slew, coef_load)
delay_pred = (weight * lut_delay).sum(dim=(1, 2)) \
             + self.correction_mlp(feat).squeeze(-1) * 0.2
```

8.3 LEVEL-ORDERED PROPAGATION

In the second stage the pins are visited level by level, in the order computed once per design with Kahn's algorithm (Fig 8.3). At each level the arrival time is updated by Eq. 3.4 over all incoming net and cell arcs, and slew is propagated along the same arcs. Primary inputs are never the destination of an arc, so a small network predicts a starting arrival time for every pin from its own features, with a softplus so that it cannot be negative.

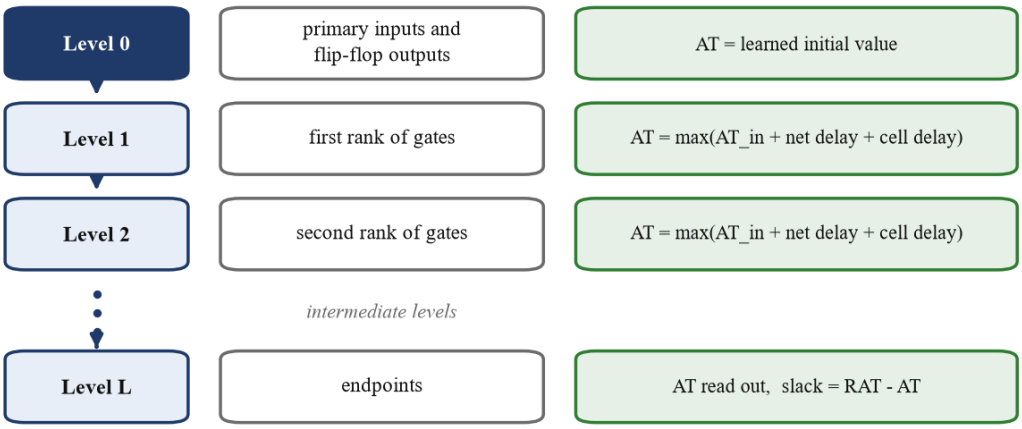


Fig 8.3 Level-ordered propagation of arrival time

```
for lvl in levels:                        # topological order
    ne = net_by_level[lvl]
    cand_at = AT[net_src[ne]] + net_delay[ne]
    AT = torch.maximum(AT, AT.index_reduce(0, net_dst[ne], cand_at, "amax"))
    ce = cell_by_level[lvl]
    cd, cs  = self.cell_lut(cell_feat[ce], lut_delay[ce], lut_slew[ce])
    cand_at = AT[cell_src[ce]] + cd.clamp(min=0.001)
    AT = torch.maximum(AT, AT.index_reduce(0, cell_dst[ce], cand_at, "amax"))
```

8.4 LOSS FUNCTION AND TRAINING

The network is trained on four quantities at once (Eq. 8.2). Arrival time is weighted most heavily; slew is supervised because it indexes the Liberty table; the net and cell delays are intermediate quantities and are given a smaller weight.

$$L = 2.0 \cdot \mathrm{MSE}(\mathrm{AT}) + \mathrm{MSE}(\mathrm{SLEW}) + 0.3 \cdot \mathrm{MSE}(\mathrm{d_net}) + 0.3 \cdot \mathrm{MSE}(\mathrm{d_cell}) \quad (8.2)$$

During training the propagation can use either the true delays or the delays the network has just predicted. True delays keep early training stable; predicted delays are what happens at test time. The probability of using the true delays is therefore lowered linearly over the run (Eq. 8.3, epoch e of E), so that by the end the network practises the conditions it will be tested under.

$$\mathrm{p_true}(e) = 1 - (e - 1) \,/\, (E - 1) \quad (8.3)$$

Table 8.1 Training configuration of the final model

Setting	Value
Hidden dimension / message-passing layers	32 / 3
Liberty table resolution	7 × 7
Trainable parameters	40,730
Optimiser	Adam, learning rate 2e-3, weight decay 1e-4
Learning-rate schedule	Reduce on plateau, factor 0.5
Gradient clipping	Maximum norm 1.0
Epochs / early-stopping patience	500 / 25 evaluations
Model selection	Mean of the per-design arrival-time R^2

8.5 EXPERIMENTS

Three candidate improvements were tested, each as a single change against the same baseline: using the SDC input delay directly as the arrival time of primary inputs (fix 1), normalising the topological level by the depth of each design (fix 2), and weighting the arrival-time loss towards pins with a large arrival time (fix 3). Together they helped jpeg_encoder and y_huff and hurt spm, xtea and synth_ram, for a lower mean even after 500 epochs (Table 8.2, Fig 8.4), so all three were reverted. Section 9.4 shows why fix 1 could not have worked.

Table 8.2 Experiments: mean arrival-time R^2 on the seven test designs

Run	Change	Epochs	Best epoch	Mean R^2
Final model	none	500	370	0.8988
Run C	fixes 1 + 2 + 3	300	300	0.8787
Run D	fixes 1 + 2 + 3	500	400	0.8919
Cycle 3	fixes 1 to 7 at once	—	—	0.869

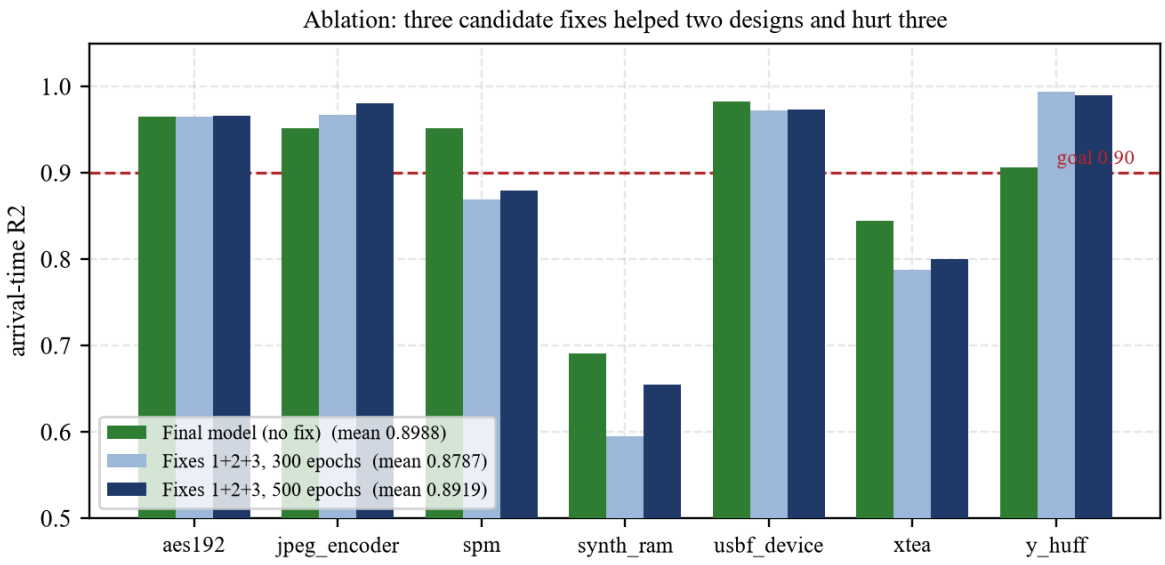


Fig 8.4 Per-design effect of the candidate fixes

CHAPTER 9: RESULTS AND TESTING

9.1 MEASURES USED

Accuracy is measured by the coefficient of determination R^2 between predicted values $\hat{y}$ and true values y over the evaluated pins (Eq. 9.1), and by the mean absolute error in nanoseconds (Eq. 9.2). $R^2 = 1$ is a perfect prediction and $R^2 = 0$ is no better than predicting the mean.

$$R^2 = 1 - \Sigma\,(y_i - \hat{y}_i)^2 \,/\, \Sigma\,(y_i - \bar{y})^2 \tag{9.1}$$

$$\mathrm{MAE} = (1/n)\,\Sigma\,|\,y_i - \hat{y}_i\,| \tag{9.2}$$

The headline figure is the mean of the seven per-design R^2 values. Pooling all pins instead would be dominated by the largest designs, which contribute more than a million evaluated pins each against about four thousand for the smallest, and could hide a failure on a small design. The goal set at the start of the phase was $R^2 \geq 0.90$ on every test design.

9.2 TRAINING BEHAVIOUR

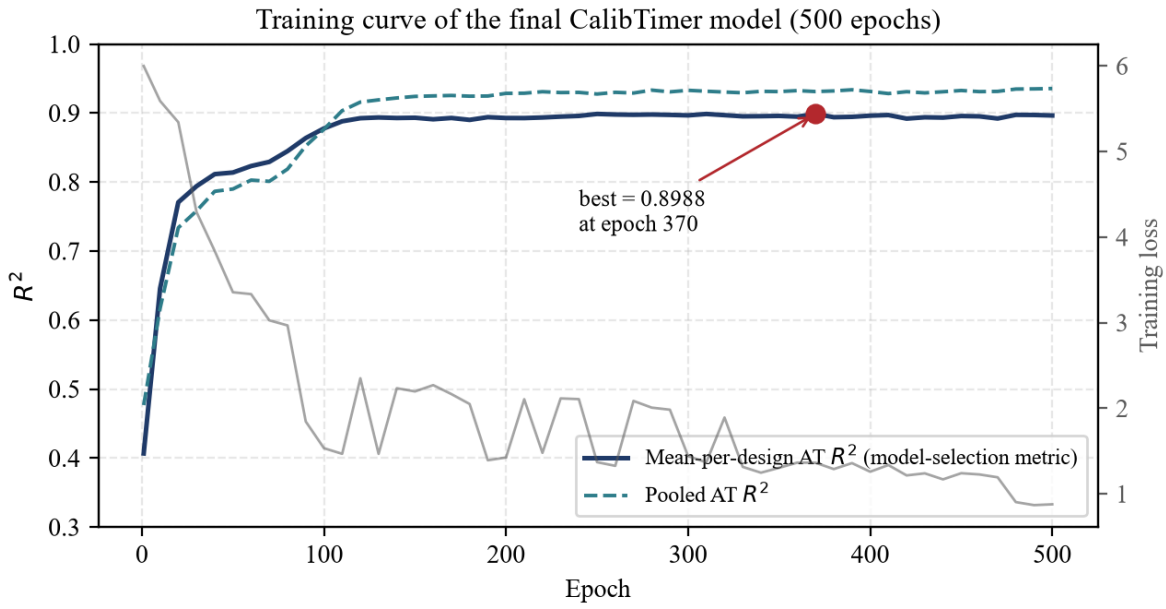


Fig 9.1 Training curve of the final model

The model reaches 0.878 after 100 epochs and 0.894 after 130, then improves very slowly to 0.8988 at epoch 370 (Fig 9.1). The pooled curve keeps rising after the per-design curve has

flattened, which is the effect described above and the reason the per-design mean was used to select the model.

9.3 ACCURACY ON THE UNSEEN TEST DESIGNS

Table 9.1 Results on the seven unseen test designs

Design	Evaluated pins	AT R²	MAE (ns)	Within 0.25 ns	Slack R²
usbf_device	235,855	0.9824	0.0367	96.6 %	0.9951
aes192	1,260,521	0.9654	0.0455	98.2 %	0.9753
spm	4,206	0.9515	0.0314	94.2 %	0.9961
jpeg_encoder	1,281,201	0.9513	0.0337	99.0 %	0.9836
y_huff	166,619	0.9063	0.0457	97.1 %	0.9737
xtea	31,743	0.8441	0.0441	98.4 %	0.9883
synth_ram	96,105	0.6909	0.2825	95.3 %	0.9725
Mean of the seven		0.8988			

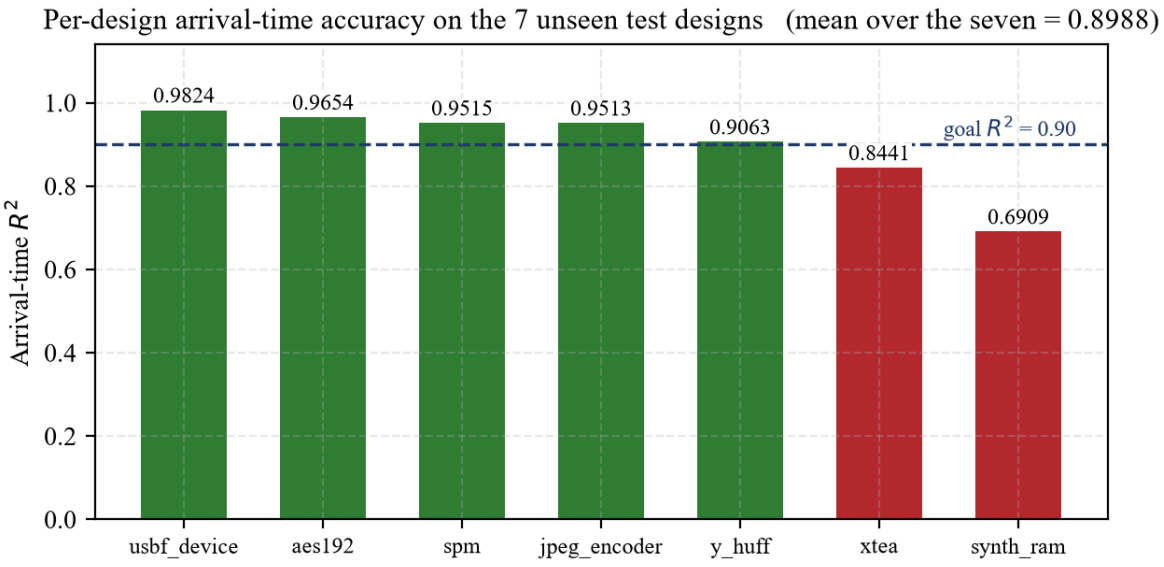


Fig 9.2 Arrival-time R² on each unseen test design

Five of the seven designs exceed the goal and four exceed 0.95 (Table 9.1, Fig 9.2). In absolute terms the error is small: on six of the seven designs the mean error is between 0.031 ns and 0.046 ns, and between 94 and 99 per cent of the pins of every design are predicted to within a quarter of a nanosecond (Fig 9.4). The slack column is not independent evidence: slack is the exact required time minus the predicted arrival time, so its error is the arrival-time error with the sign reversed, and its R² is higher only because slack has a larger spread.

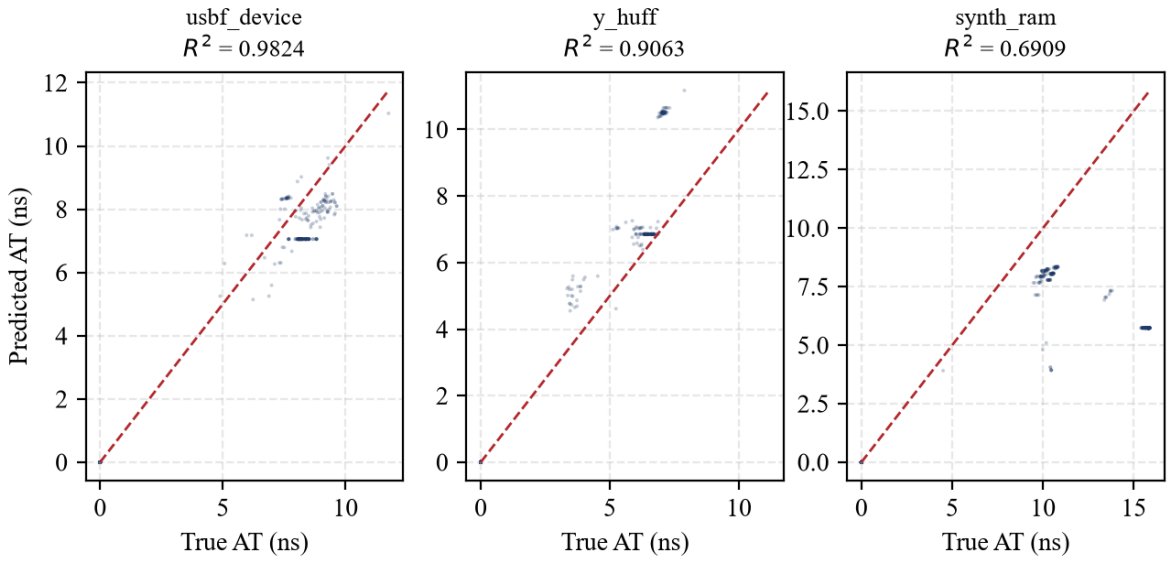


Fig 9.3 Predicted against true arrival time for three test designs

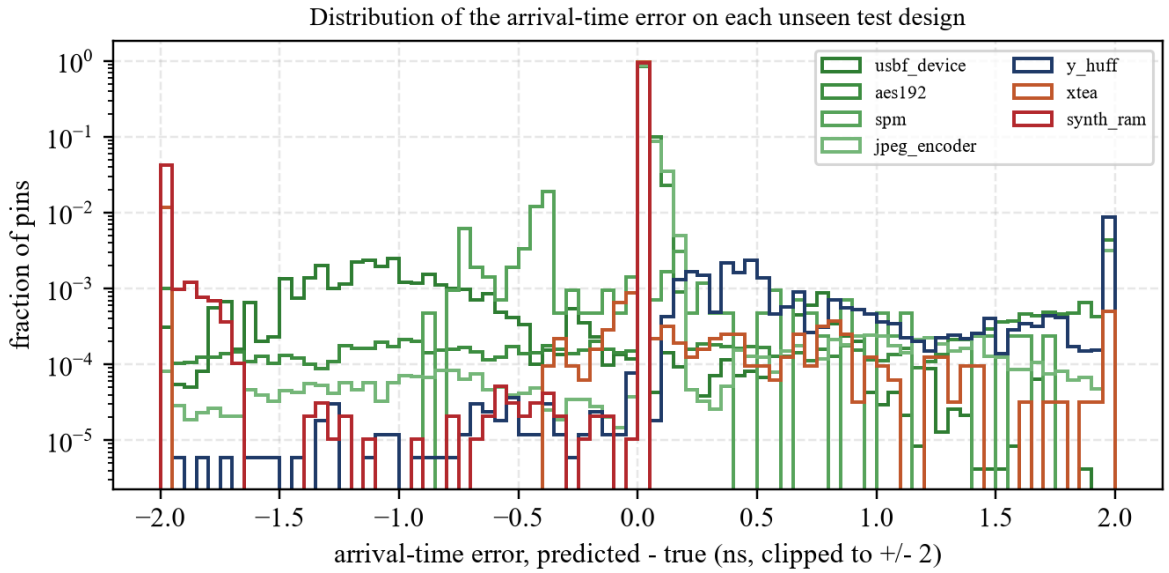


Fig 9.4 Distribution of the arrival-time error on each test design

Fig 9.5 places the predictions back on the real die of usbf_device: the true and the predicted arrival time of 89,107 placed pins, and their difference. The two maps are almost indistinguishable, and the error is spread thinly over the whole die rather than concentrated in one region.

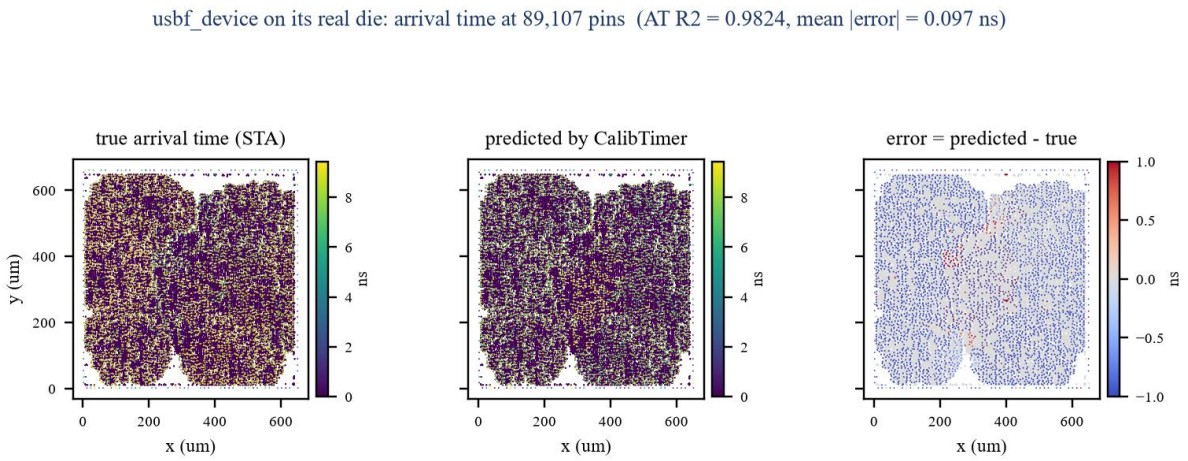


Fig 9.5 Arrival time on the real die of usbf_device: true, predicted and error

9.4 THE FAILURE CASE AND WHAT WAS DONE

Two designs miss the goal: xtea at 0.8441 and synth_ram at 0.6909. The working explanation during training was that they lay outside the training distribution, xtea in its input delay and synth_ram in its logic depth, and fixes 1 and 2 of Section 8.5 were aimed at exactly those. Neither helped. The tests of Section 9.8 showed why: both explanations were wrong, and one of the fixes had been acting on an input that was always zero.

What the tests found

- Depth is not the cause. synth_ram has only 24 topological levels, while the training designs reach 146 (picorv32a), 128 (salsa20) and 122 (blabla); xtea's 98 levels are also inside the training range.
- The input-delay feature was dead. Every SDC file states its input delays in the form set_input_delay 5.2060 -clock [get_clocks {clock}] -add_delay [get_ports {data_in1[0]}]. The parser's pattern skipped forward only to the first square bracket, which belongs to get_clocks, so it never reached get_ports. It matched none of the 4,317 input-delay constraints in the 27 designs, and node feature 11 was zero on every pin of every design. Fix 1 therefore set the arrival time of every primary input to exactly 0 ns.
- The failures line up with input delay. The true arrival time at a primary input equals its SDC input delay exactly. The five designs that pass have input delays of 2.0 to 3.2 ns; the two that fail have the two largest, xtea 5.2 ns and synth_ram 10.0 ns (Fig 9.6a, Table 9.2).

Table 9.2 Arrival time at the primary-input pins (medians) and the overall error bias

Design	SDC input delay (ns)	True AT at inputs (ns)	Predicted AT at inputs (ns)	Mean error (ns)	AT R^2
spm	2.000	2.000	1.614	-0.0168	0.9515
y_huff	2.270	2.270	3.850	+0.0451	0.9063
aes192	2.474	2.474	7.508	+0.0261	0.9654
usbf_device	2.920	2.920	1.410	-0.0306	0.9824
jpeg_encoder	3.156	3.156	7.801	+0.0301	0.9513
xtea	5.206	5.206	2.727	-0.0357	0.8441
synth_ram	10.000	10.000	3.137	-0.2824	0.6909

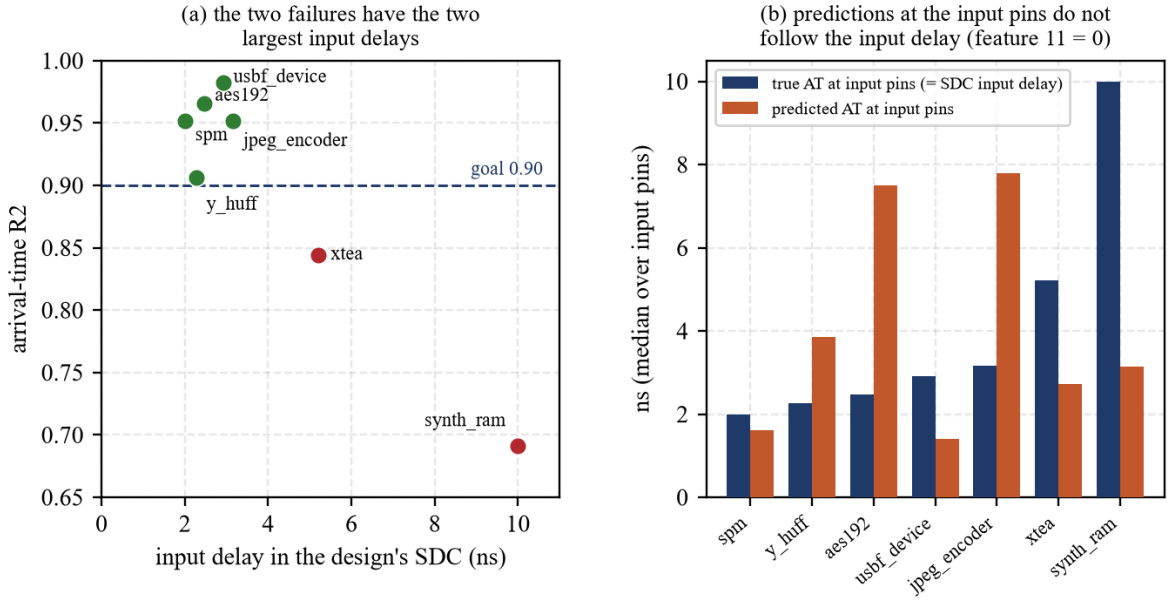


Fig 9.6 The failures against the input delay, and the arrival time predicted at the input pins

For synth_ram the explanation holds. Its inputs truly arrive at 10.0 ns, but with feature 11 at zero the network has no way to know this and starts them at about 3.1 ns. It is the only test design with a systematic bias, −0.28 ns against at most 0.05 ns on the other six, which is what launching every input-driven path about seven nanoseconds early would produce. For xtea the evidence is weaker: its bias is as small as that of the passing designs, so its input delay cannot be the whole story, and the cause of its lower score is not yet established.

A corrected pattern was written and checked against every SDC file; it reads all 4,317 constraints, including bus ports such as data_in1[0].

```
# before: stops at the first '[', which is [get_clocks ...]
r'set_input_delay\s+([\d.]+)[^\[]*\[get_ports\s*\{?\s*([^\}\]]+?)\s*\}?\s*\]'
# after: skips any options, including [get_clocks {...}]
r'set_input_delay\s+([\d.]+)\b.*?\[get_ports\s+(?:\{([^}]+)\}|([^\]\s]+))\s*\]'
```

Because the feature changes the data the model is trained on, the corrected parser requires a new training run; that run is the first item of future work (Section 10.7). The numbers reported in this chapter are those of the model as trained.

9.5 CONFORMAL CALIBRATION OF THE SLACK PREDICTION

A single predicted number is of limited use to a designer deciding whether a path is safe; what is needed is an interval that contains the true value with a stated probability. Split conformal prediction [14] provides one without any assumption about the distribution of the errors (Fig 9.7). The trained model is run on the calibration designs, which were held out of training for this purpose; the absolute slack error s_i of every calibration pin is computed; and the half-width q of the interval is the finite-sample-corrected quantile of those errors (Eq. 9.3), where n is the number of calibration pins and α the permitted miss rate. Every new prediction then carries the interval of Eq. 9.4.

$$q = \text{the } \lceil (n+1)(1-\alpha) \rceil\text{-th smallest of } s_i = |\, \text{slack}_i - \widehat{\text{slack}}_i \,| \tag{9.3}$$

$$C(x) = [\, \widehat{\text{slack}}(x) - q \,,\ \widehat{\text{slack}}(x) + q \,] \tag{9.4}$$

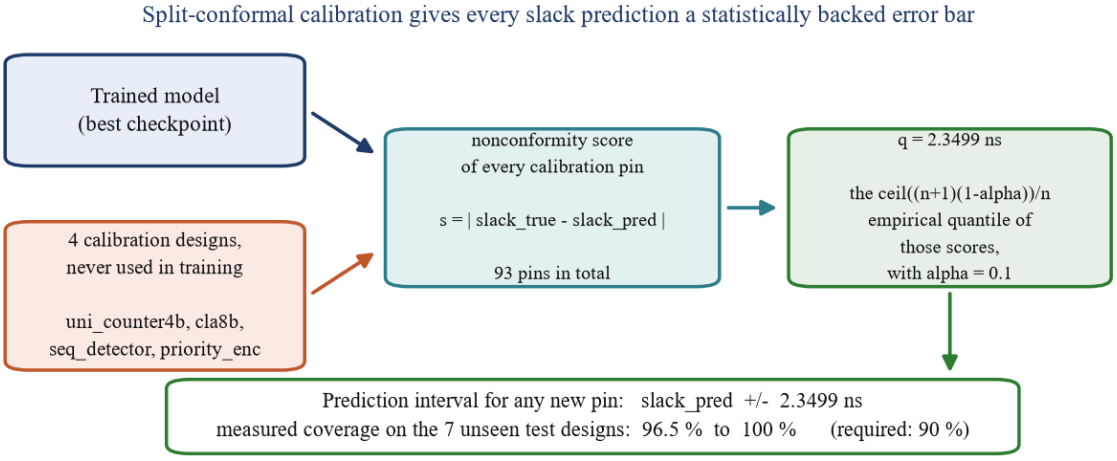


Fig 9.7 The split-conformal calibration procedure

With α = 0.1, that is a required coverage of 90 per cent, calibration on 93 pins from four calibration designs gave q = 2.3499 ns. The fifth, gcd, was left out because its graph is not

acyclic (test T3). The coverage of the interval was then measured on the seven unseen designs, over 3,076,250 pins (Table 9.3, Fig 9.8).

Table 9.3 Measured coverage of the calibrated slack interval (± 2.3499 ns)

Design	Pins checked	Required	Measured coverage
aes192	1,260,521	90.0 %	99.7 %
jpeg_encoder	1,281,201	90.0 %	99.7 %
spm	4,206	90.0 %	100.0 %
synth_ram	96,105	90.0 %	96.5 %
usbf_device	235,855	90.0 %	100.0 %
xtea	31,743	90.0 %	98.8 %
y_huff	166,619	90.0 %	99.2 %
All test pins	3,076,250	90.0 %	99.6 %

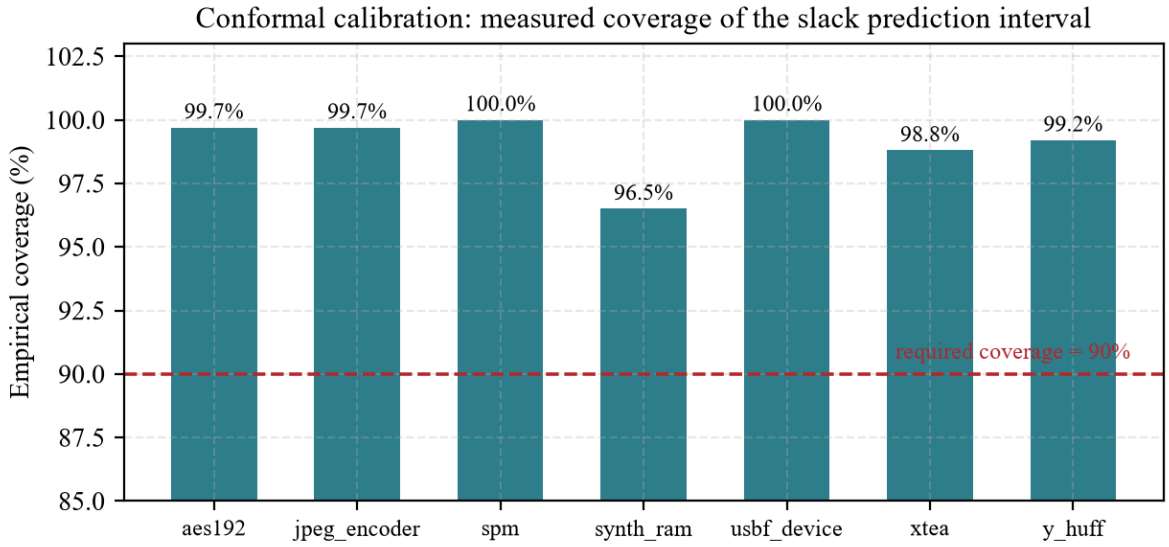


Fig 9.8 Measured coverage of the calibrated interval on each test design

The calibration holds on every design, including synth_ram, the design the model predicts worst, which is precisely where a designer needs the guarantee most. Coverage well above 90 per cent means the interval is conservative, wider than it strictly needs to be, because the four calibration designs are small circuits of a few hundred cells; a larger and more varied calibration pool would tighten it.

9.6 SPEED

The point of a predictor is to avoid routing. Table 9.4 gives the time CalibTimer takes on each test design on the laptop of Table 4.2, measured from the design files to the predicted arrival time at every pin, beside the time the reference work [1] reports for its OpenROAD routing plus timing analysis of the same designs. The machines differ, so the ratio is indicative only; the network itself evaluates a complete design in 8 to 160 ms.

Table 9.4 Time to predict a whole design, against routing and STA

Design	Pins	Parse (s)	Tensors (s)	Network (ms)	Total (s)	Routing + STA in [1] (s)
spm	4,728	0.05	4.79	8	4.86	18.2
xtea	40,810	0.66	0.40	37	1.10	178.9
synth_ram	111,277	1.57	0.74	57	2.37	461.3
y_huff	202,190	2.91	1.62	35	4.56	1177.4
usbf_device	281,685	4.23	2.36	47	6.63	1546.0
aes192	1,424,098	18.71	12.64	151	31.51	1786.6
jpeg_encoder	1,485,105	20.46	14.85	160	35.46	4261.3

The first design processed (spm) includes the one-time loading of the Liberty library, about 4.5 s. Excluding it, prediction takes from about one second for xtea to 35 seconds for jpeg_encoder, against 179 s and 4,261 s respectively for routing and analysis.

9.7 COMPARISON WITH REFERENCE PAPERS

Table 9.5 places CalibTimer beside the two papers that report results on the same seven test designs. The figures of the other papers are as their authors report them. The comparison is indicative: the reference work computes R^2 over all graph nodes, whereas CalibTimer evaluates only the pins that carry a real timing label, and E2ESlack trains on 23 circuits against 15 here.

Table 9.5 Arrival-time R^2 on the seven test designs: this work and the reference papers

Design	Guo et al. [1]	E2ESlack [2]	This work
jpeg_encoder	0.8820	0.9169	0.9513
usbf_device	0.9252	0.9202	0.9824
aes192	0.8605	0.9530	0.9654
xtea	0.9135	0.9534	0.8441
spm	0.8975	0.9605	0.9515
y_huff	0.9256	0.8231	0.9063
synth_ram	0.8656	0.8911	0.6909
Mean	0.8957	0.9169	0.8988
Training designs	14	23	15

CalibTimer's mean is close to that of the reference work and below that of E2ESlack. Per design it is the best of the three on jpeg_encoder, usbf_device and aes192, the three largest designs, and the weakest on xtea and synth_ram, the two designs with the largest input delays, consistent with the defect of Section 9.4.

9.8 TESTING

The tests in Table 9.6 were run against the code and the trained model of the final run, on the real design files. A test passes when the actual output agrees with the expected output.

Table 9.6 Test cases

ID	Test condition	Expected	Actual	Result
T1	Leakage guard on the 27 design folders	15 train, 7 test, no overlap	27 found; 15 train, 7 test; 0 overlap	Pass
T2	A test design injected into the training list	Assertion stops the run	Assertion fired: LEAKAGE GUARD	Pass
T3	Every design parses into an acyclic graph	27 of 27 acyclic	26 of 27; gcd contains a cycle	Fail (gcd)
T4	Feature widths of the tensors	12 node, 4 net-edge	12 node, 4 net-edge	Pass
T5	Clock period read from the SDC	Present for sequential, absent for combinational designs	spm 10, xtea 26.03, synth_ram 50; cla8b and priority_enc none	Pass
T6	Liberty tables loaded	nand2_1 A→Y found, 7 × 7	1241 arcs; found; 7 × 7	Pass
T7	Checkpoint reloaded with the run's own source snapshot	Loads without error	Loads	Pass
T8	Checkpoint reloaded with the current source	Loads	Width mismatch 247 vs 248 (a slew input was added later)	Fail
T9	Re-measure AT R^2 from the saved model	Equal to the report	spm 0.9515, xtea 0.8441 (reported 0.9515, 0.8441)	Pass
T10	Primary inputs not frozen at 0 ns	Predicted AT > 0	min 0.31 ns (spm), 1.53 ns (xtea)	Pass
T11	Input delays read from the SDC files	4,317 of 4,317	0 of 4,317; feature 11 = 0 everywhere. Corrected pattern: 4,317 of 4,317	Fail, fixed
T12	Conformal coverage on each test design	≥ 90 %	96.5 % to 100 %	Pass

Three tests failed, and each failure is informative. T3 explains why gcd was excluded from calibration. T8 shows that a stored source snapshot is not optional: the current code can no longer load the reported model, and without the snapshot of the run the result could not have been reproduced in T9. T11 found the defect of Section 9.4, which had survived every training run because the network trains without error on a feature that is constant.

CHAPTER 10: CONCLUSION AND DISCUSSION

10.1 OVERALL ANALYSIS OF THE INTERNSHIP

The internship covered a complete research cycle three times, at increasing depth. The first phase built a working pipeline and a convolutional network evaluated with a metric suited to an unbalanced problem. The second tested a popular idea fairly and found it did not hold here, which led to a change of direction rather than more tuning. The third was a genuine research problem: a model matched to the structure of the data, an input pipeline written from industrial formats, an evaluation strict enough for the numbers to mean something, a calibration step that gives them a guarantee, and a set of tests that found what training alone had not.

10.2 MAIN RESULTS

- A one-dimensional SE-ResNet classifies the PhysioNet/CinC 2017 recordings with 80.5 % accuracy and 0.730 macro F1.
- Eighteen two-dimensional networks and two residual translations on time-frequency images reach at best 0.660 macro F1, below the one-dimensional network.
- CalibTimer, with 40,730 parameters, predicts pin-level arrival time on seven unseen sky130 designs with a mean R^2 of 0.8988; four exceed 0.95, and 94 to 99 % of the pins of every design are within 0.25 ns.
- Split-conformal calibration gives a slack interval of ± 2.35 ns whose measured coverage is 96.5 to 100 % on every test design, against 90 % required.
- The network evaluates a whole design in 8 to 160 ms; from design files to prediction takes 1 to 35 s, against 18 to 4,261 s reported for routing and timing analysis.
- Testing found that the input-delay feature was always zero because of a parsing defect, which explains the synth_ram result; a corrected parser reads all 4,317 constraints.

10.3 ENVIRONMENT AND SUPPORT AT IIIT VADODARA

The working environment was that of a research group. The supervisor, Dr. Kamal Kishor Jha, set the direction, reviewed the results at each stage, and insisted that every reported number be reproducible and that test data never leak into training. That insistence is why every run in this report was stored with its own source snapshot, which in the end made it

possible to reproduce the reported model exactly and to find the defect described in Chapter 9.

10.4 PROBLEMS ENCOUNTERED AND THEIR SOLUTIONS

Table 10.1 Problems encountered and solutions

Problem	Solution
2-D image networks did not beat the 1-D network	Fair comparison on a common split; project re-scoped to timing prediction
Class imbalance in the ECG data	Focal loss; macro F1 as the comparison metric
Timing endpoints identified wrongly	Cell-edge drivers added to the exclusion set; earlier results discarded
True arrival times parsed but then replaced by an estimate	Real .at.sdf values written into the labels; earlier results discarded
Calibration designs leaking into training	Fixed test and calibration sets with assertions in the training script
Loops in the graph through flip-flops	No D → Q cell edge for sequential cells
Primary inputs frozen at an arrival time of 0	Learned initial arrival time per pin
Delay predictions and arrival-time loss interfering	Predicted delays detached before propagation
Out-of-memory error on the largest designs	Per-level results collected and written once after the loop
Pooled R^2 hiding failures on small designs	Mean of per-design R^2 used for model selection
Current code cannot load the reported model	Source snapshot stored with every run
Input-delay feature always zero	Corrected SDC pattern, verified on all 4,317 constraints; retraining planned

10.5 SKILLS AND LEARNING OUTCOMES

- Reading research papers critically and checking how a reported number was obtained.
- Designing and training one- and two-dimensional convolutional networks, with residual and attention blocks and loss functions for unbalanced data.
- Understanding the physical design flow and static timing analysis.
- Parsing industrial formats: Verilog, DEF, SPEF, SDF, SDC and Liberty.
- Implementing a graph neural network from first principles in PyTorch.
- Applying split conformal prediction to obtain a distribution-free interval.
- Writing tests that check a result rather than assume it, and keeping a record of experiments so that every number can be traced to its files.

10.6 LIMITATIONS

- The model was trained with the input-delay feature at zero; the effect of the corrected parser has not yet been measured.
- The cause of the lower score on xtea is not established.
- Slew and net delay are predicted far less accurately than arrival time.
- The calibration pool is four small designs, which makes the interval conservative, and the fifth, gcd, has a cyclic graph.
- All results are for one process, sky130, at one corner.

10.7 FUTURE SCOPE

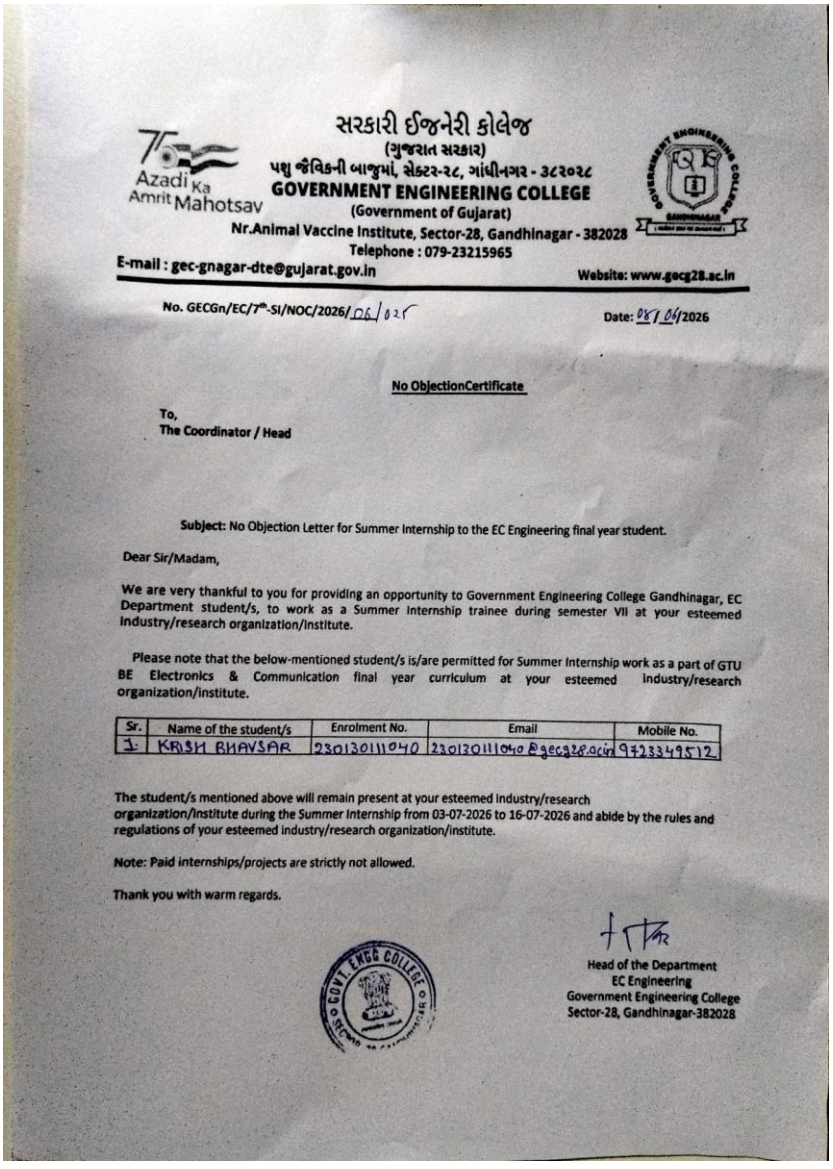
- Retrain with the corrected input-delay parser and measure the change on every test design, xtea and synth_ram in particular.
- Investigate the remaining error on xtea.
- Re-synthesise gcd so that its graph is acyclic, and enlarge the calibration pool with larger and more varied designs to tighten the interval.
- Predict the required arrival time directly, to report total and worst negative slack for the whole design.
- Add placement density and congestion as inputs, and test transfer to another technology node.

10.8 SUMMARY

The internship connected signal processing, machine learning and VLSI design, three subjects of my Electronics and Communication Engineering course, in one piece of work. It began with convolutional networks on a heart signal and ended with a graph neural network that predicts the timing of a digital circuit before it is routed, with a calibrated interval around every slack prediction. It taught me that a result is only as good as the way it is evaluated and tested: the most useful finding of the last phase came not from training a better model but from testing the one I had.

APPENDIX 1: NO OBJECTION CERTIFICATE FROM THE DEPARTMENT

No Objection Certificate No. GECGn/EC/7th-SI/NOC/2026/06/025, dated 08/06/2026, issued by the Head of the Department of Electronics and Communication Engineering, Government Engineering College, Gandhinagar, for the summer internship from 03-07-2026 to 16-07-2026.



સરકારી ઈજનેરી કોલેજ
(ગુજરાત સરકાર)
પશુ રસીકેન્દ્ર ની બાજુમાં, સેક્ટર-૨૮, ગાંધીનગર - ૩૮૨૦૨૮
GOVERNMENT ENGINEERING COLLEGE
(Government of Gujarat)
Nr.Animal Vaccine Institute, Sector-28, Gandhinagar - 382028
Telephone : 079-23215965
E-mail : gec-gnagar-dte@gujarat.gov.in
Website: www.gecg28.ac.in

No. GECGn/EC/7th-SI/NOC/2026/06/025 Date: 08/06/2026

No ObjectionCertificate

To,
The Coordinator / Head

Subject: No Objection Letter for Summer Internship to the EC Engineering final year student.

Dear Sir/Madam,

We are very thankful to you for providing an opportunity to Government Engineering College Gandhinagar, EC Department student/s, to work as a Summer Internship trainee during semester VII at your esteemed industry/research organization/institute.

Please note that the below-mentioned student/s is/are permitted for Summer Internship work as a part of GTU BE Electronics & Communication final year curriculum at your esteemed industry/research organization/institute.

Sr.	Name of the student/s	Enrolment No.	Email	Mobile No.
1.	KRISH BHAVSAR	230130111040	230130111040@gecg28.ac.in	9723349512

The student/s mentioned above will remain present at your esteemed industry/research organization/institute during the Summer Internship from 03-07-2026 to 16-07-2026 and abide by the rules and regulations of your esteemed industry/research organization/institute.

Note: Paid internships/projects are strictly not allowed.

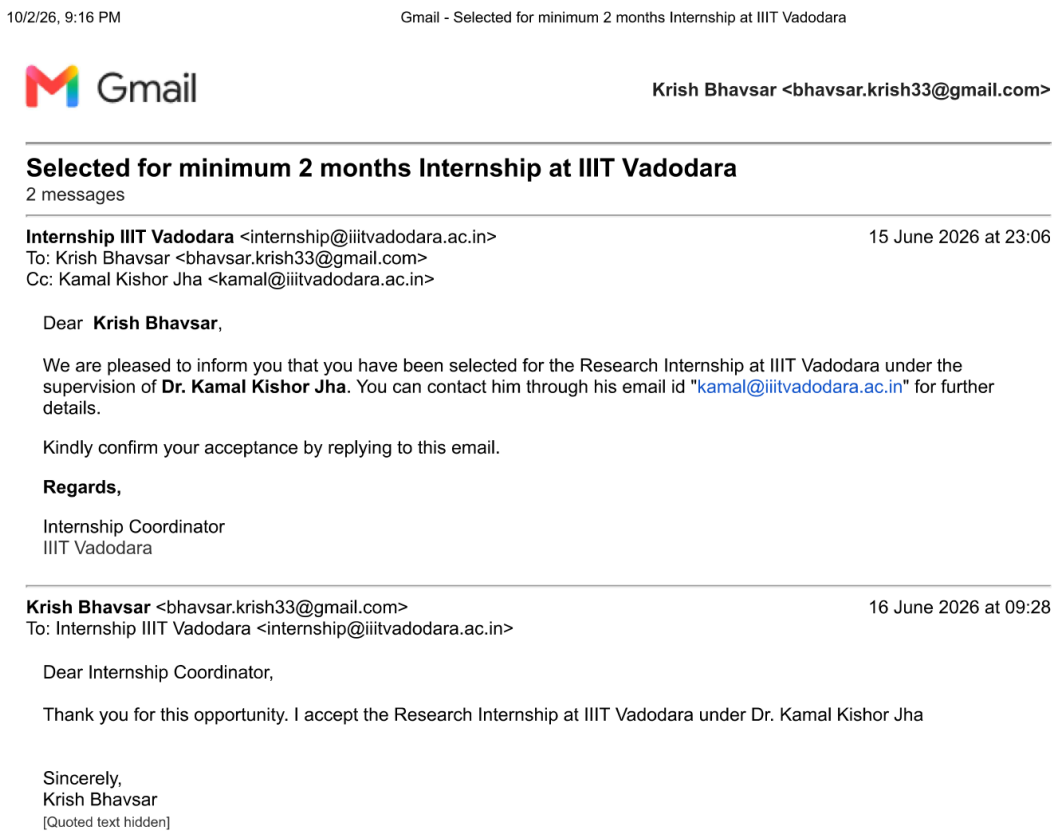
Thank you with warm regards.

Head of the Department
EC Engineering
Government Engineering College
Sector-28, Gandhinagar-382028

Fig A1.1 No Objection Certificate issued by the EC Department

APPENDIX 2: SELECTION (JOINING) EMAIL FROM IIIT VADODARA

Selection email from the internship coordinator of IIIT Vadodara, dated 15 June 2026, copied to Dr. Kamal Kishor Jha, and the acceptance sent on 16 June 2026.



10/2/26, 9:16 PM Gmail - Selected for minimum 2 months Internship at IIIT Vadodara

Gmail **Krish Bhavsar <bhavsar.krish33@gmail.com>**

Selected for minimum 2 months Internship at IIIT Vadodara

2 messages

Internship IIIT Vadodara <internship@iiitvadodara.ac.in> 15 June 2026 at 23:06
To: Krish Bhavsar <bhavsar.krish33@gmail.com>
Cc: Kamal Kishor Jha <kamal@iiitvadodara.ac.in>

Dear **Krish Bhavsar**,

We are pleased to inform you that you have been selected for the Research Internship at IIIT Vadodara under the supervision of **Dr. Kamal Kishor Jha**. You can contact him through his email id "kamal@iiitvadodara.ac.in" for further details.

Kindly confirm your acceptance by replying to this email.

Regards,

Internship Coordinator
IIIT Vadodara

Krish Bhavsar <bhavsar.krish33@gmail.com> 16 June 2026 at 09:28
To: Internship IIIT Vadodara <internship@iiitvadodara.ac.in>

Dear Internship Coordinator,

Thank you for this opportunity. I accept the Research Internship at IIIT Vadodara under Dr. Kamal Kishor Jha

Sincerely,
Krish Bhavsar
[Quoted text hidden]

Fig A2.1 Selection email and acceptance

APPENDIX 3: WEEKLY RECORD OF INTERNSHIP (ANNEXURE 1)

The signed weekly records for Week 1 (03/07/2026 to 09/07/2026) and Week 2 (10/07/2026 to 16/07/2026) are attached on the following pages.

GUJARAT TECHNOLOGICAL UNIVERSITY
(Established under Gujarat Act No. 20 of 2007)
ગુજરાત ટેકનોલોજીકલ યુનિવર્સિટી
(ગુજરાત અધિનિયમ ક્રમાંક: ૨૦/૨૦૦૨ દ્વારા સ્થાપિત)

Annexure 1
Enrollment no: 230130111040

STUDENT'S WEEKLY RECORD OF INTERNSHIP

NAME OF STUDENT: Krish Bhavsar

DIARY OF THE WEEK: Dt: 03/07/2026 **TO** 09/07/2026

DEPARTMENT: Electronics and Communication Engineering **SEM:** VII

NAME OF THE ORGANISATION: Indian Institute of Information Technology Vadodara

NAME OF THE PLANT/SECTION/DEPARTMENT: Dept. of Electronics and Communication Engg.

NAME OF OFFICER INCHARGE OF THE PLANT/SECTION/DEPARTMENT: Dr. Kamal Kishor Jha

DESCRIPTION OF THE WORK DONE IN BRIEF

Project: VLSI and AI-based ECG Arrhythmia Classification using 1D and 2D Convolutional Neural Networks

1. Literature Study and Problem Formulation

- Studied the clinical background of cardiac arrhythmia and the morphology of the ECG waveform, along with the four-class labelling scheme used in the PhysioNet/CinC Challenge 2017 dataset: Normal (N), Atrial Fibrillation (A), Other rhythm (O) and Noisy (~).
- Reviewed literature on deep-learning-based ECG classification, covering both 1D-CNN architectures operating directly on the raw waveform and 2D-CNN architectures operating on time-frequency image representations of the signal.
- Configured the development environment with Python 3.11, TensorFlow/Keras, NumPy, SciPy and scikit-learn under WSL Ubuntu, together with the WFDB package for reading PhysioNet-format ECG records.

2. Dataset Preparation - PhysioNet/CinC Challenge 2017

- Acquired and organised the PhysioNet/CinC Challenge 2017 database of 8,528 single-lead ECG recordings labelled into four classes: Normal (N), Atrial Fibrillation (A), Other rhythm (O) and Noisy (~).
- Implemented the pre-processing pipeline: band-pass filtering and baseline-wander removal, amplitude normalisation, R-peak detection and fixed-length windowing.

3. 1D-CNN Baseline Implementation and Verification

- Implemented a baseline 1D-CNN of stacked convolution, batch-normalisation, ReLU and max-pooling blocks followed by global pooling and a soft-max layer, and trained it on the pre-processed waveforms for the N / A / O / ~ four-class classification task.
- Translated the trained model into nine hand-coded Verilog RTL modules (MAC unit, 1D convolution engine, max-pooling, global average pooling, dense layers, argmax classifier, top-level wrapper and an AXI4-Lite interface) and verified the design bit-perfect against a NumPy reference model.
- Identified and fixed a signed/unsigned comparison bug in the argmax classifier's part-select logic found during

verification.

4. 2D-CNN Variant using Time-Frequency Representations

- Converted the same ECG recordings into 2D time-frequency representations - spectrograms, scalograms and DWT (discrete wavelet transform) maps - using STFT/CWT/DWT.
- Implemented an SE-ResNet architecture on these 2D representations for the same N / A / O / ~ classification task and compared it against the 1D-CNN baseline.
- Observed the 2D-CNN (SE-ResNet) reach a macro F1-score of approximately 0.66, against approximately 0.76 for the 1D-CNN baseline, and documented the trade-off between the two approaches.

5. Hardware Deployment on PYNQ-Z2

- Targeted the verified 1D-CNN accelerator for deployment on the Xilinx PYNQ-Z2 FPGA board and compiled the week's 1D- and 2D-CNN results into a consolidated report.

GUJARAT TECHNOLOGICAL UNIVERSITY
(Established under Gujarat Act No. 20 of 2007)
ગુજરાત ટેકનોલોજીકલ યુનિવર્સિટી
(ગુજરાત અધિનિયમ ક્રમાંક: ૨૦/૨૦૦૨ દ્વારા સ્થાપિત)

TOTAL HOURS: 42

SIGNATURE OF STUDENT

✦ **The above entries are correct and the grading of work done by Trainee is EXCELLENT / VERY GOOD / GOOD / FAIR / BELOW AVERAGE / POOR**

Signature of Faculty Mentor	**Signature of officer-in-charge of Dept. / Section / Plant**
Date:	**Date:**

✦ **Grading of Work, for trainee may be given depending upon your judgement about his Punctuality, Regularity, Sincerity, Interest taken, Work done etc.**

GUJARAT TECHNOLOGICAL UNIVERSITY
(Established under Gujarat Act No. 20 of 2007)
ગુજરાત ટેકનોલોજીકલ યુનિવર્સિટી
(ગુજરાત અધિનિયમ ક્રમાંક: ૨૦/૨૦૦૨ દ્વારા સ્થાપિત)

Annexure 1
Enrollment no: 230130111040

STUDENT'S WEEKLY RECORD OF INTERNSHIP

NAME OF STUDENT: Krish Bhavsar

DIARY OF THE WEEK: Dt: 10/07/2026 **TO** 16/07/2026

DEPARTMENT: Electronics and Communication Engineering **SEM:** VII

NAME OF THE ORGANISATION: Indian Institute of Information Technology Vadodara

NAME OF THE PLANT/SECTION/DEPARTMENT: Dept. of Electronics and Communication Engg.

NAME OF OFFICER INCHARGE OF THE PLANT/SECTION/DEPARTMENT: Dr. Kamal Kishor Jha

DESCRIPTION OF THE WORK DONE IN BRIEF

Project: GCN-based Pre-Routing Timing Slack Prediction (TimingGNN)

1. Transition to GCN-based Timing Prediction

- Moved from the ECG 1D-CNN accelerator work to a new research problem: predicting post-routing timing slack directly from a pre-routing netlist using a Graph Convolutional Network (GCN), to avoid the cost of full physical-design iterations.
- Studied reference literature on GNN-based EDA applications, in particular Guo et al. (DAC 2022) and Mirhoseini et al. (Nature 2021), to frame the problem and identify baseline approaches.

2. Data Pipeline Development

- Built a custom parser for DEF, Verilog and SDF files to extract circuit-graph structure and timing annotations, including support for the min::max SDF format.
- Implemented filtering of power and filler cells so that the constructed circuit graphs reflect only functionally relevant standard cells.
- Set up training data using the spm circuit (real design) together with synthetic circuits, reserving the usb circuit (unseen during training) as the generalisation benchmark.

3. Model Design and Experimentation

- Implemented the GCN-based timing slack predictor in PyTorch with configurable node-aggregation strategies: SUM+MAX pooling, attention-based aggregation and mean aggregation.
- Ran an ablation study comparing the aggregation strategies, which formed the core experimental contribution of the work, and evaluated model generalisation on the unseen usb circuit.
- Compiled the week's model-development and ablation results into a consolidated report.

GUJARAT TECHNOLOGICAL UNIVERSITY
(Established under Gujarat Act No. 20 of 2007)
ગુજરાત ટેકનોલોજીકલ યુનિવર્સિટી
(ગુજરાત અધિનિયમ ક્રમાંક: ૨૦/૨૦૦૨ દ્વારા સ્થાપિત)

TOTAL HOURS: 42

SIGNATURE OF STUDENT

✦ **The above entries are correct and the grading of work done by Trainee is EXCELLENT / VERY GOOD / GOOD / FAIR / BELOW AVERAGE / POOR**

Signature of Faculty Mentor	**Signature of officer-in-charge of Dept. / Section / Plant**
Date:	**Date:**

✦ **Grading of Work, for trainee may be given depending upon your judgement about his Punctuality, Regularity, Sincerity, Interest taken, Work done etc.**

APPENDIX 4: SELECTED CODE LISTINGS

Short extracts that show the algorithms described in the report. They are not the complete source.

A4.1 Topological levels with Kahn's algorithm

```
indeg = [0] * N
succ  = [[] for _ in range(N)]
for a, b in g["net_edges"] + g["cell_edges"]:
    succ[a].append(b); indeg[b] += 1
level = [0] * N
q = deque(i for i in range(N) if indeg[i] == 0)
while q:
    u = q.popleft()
    for v in succ[u]:
        level[v] = max(level[v], level[u] + 1)
        indeg[v] -= 1
        if indeg[v] == 0:
            q.append(v)
```

A4.2 The twelve node features

```
x = torch.tensor([[
    p["x"] / canvas_scale,  p["y"] / canvas_scale,
    float(p["kind"] == "PI"), float(p["kind"] == "IN"), float(p["kind"] == "OUT"),
    p["dist_left"] / die_w,  p["dist_right"] / die_w,
    p["dist_bottom"] / die_h, p["dist_top"] / die_h,
    float(i in endpoint_set),
    fanout_count.get(i, 0) / 10.0,
    true_input_delay.get(i, 0.0) / INPUT_DELAY_NORM_SCALE,   # feature 11
] for i, p in enumerate(pins)], dtype=torch.float)
assert x.shape[1] == NODE_FEATURES        # 12
```

A4.3 Message passing with the virtual node

```
for i, layer in enumerate(self.update_mlps):
    h = h + vn                                  # broadcast global state
    msg = self.msg_mlp(torch.cat([h[net_src], edge_h], dim=-1))
    agg = torch.zeros_like(h).index_add_(0, net_dst, msg)
    h = layer(torch.cat([h, agg], dim=-1)) + h   # residual update
    vn = vn + self.vn_update_mlps[i](h.mean(dim=0, keepdim=True))
```

A4.4 Split-conformal quantile

```
def conformal_quantile(scores, alpha):
    n = len(scores)
    k = math.ceil((n + 1) * (1 - alpha))      # finite-sample correction
    if k > n:
        raise ValueError("calibration set too small for this alpha")
    return sorted(scores)[k - 1]
```

A4.5 Test T11: reading the input delays

```
BUG = re.compile(r'set_input_delay\s+([\d.]+)[^\[]*\[get_ports\s*\{?\s*([^\}\]]+?)\s*\}?\s*\]')
FIX = re.compile(r'set_input_delay\s+([\d.]+)\b.*?\[get_ports\s+(?:\{([^}]+)\}|([^\]\s]+))\s*\]')
for each design:
    lines = [l for l in sdc.splitlines() if l.startswith('set_input_delay')]
```

```
    print(len(lines), len(BUG.findall(sdc)), len(FIX.findall(sdc)))
# total over 27 designs:  4317 lines,  0 matched (BUG),  4317 matched (FIX)
```

REFERENCES

1. Guo, Z., Liu, M., Gu, J., Zhang, S., Pan, D. Z. and Lin, Y. (2022) ‘A timing engine inspired graph neural network model for pre-routing slack prediction’, Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC).
2. Bodhe, S., Zhang, Z., Hamidizadeh, A., et al. (2025) ‘E2ESlack: an end-to-end graph-based framework for pre-routing slack prediction’, arXiv:2501.07564.
3. Bairamkulov, R. and Friedman, E. G. (2023) Graphs in VLSI. Cham: Springer.
4. Hu, Y., Li, J., et al. (2023) ‘SyncTREE: fast timing analysis for integrated circuit design through a physics-informed tree-based graph neural network’, Advances in Neural Information Processing Systems (NeurIPS).
5. Zhang, X., Zhu, B., Liu, F., Jiang, J., Wang, Z., Xu, P., et al. (2025) ‘Prerouting timing prediction across different technology nodes’, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 44(7), p. 2697 onwards.
6. Barboza, E. C., Shukla, N., Chen, Y. and Hu, J. (2019) ‘Machine learning-based pre-routing timing prediction with reduced pessimism’, Proceedings of the 56th Design Automation Conference (DAC).
7. Chang, K. and Kim, T. (2024) ‘Pre-route timing prediction and optimization with graph neural network models’, Integration, the VLSI Journal.
8. Li, J., Hu, J., Wu, Y. and Yang, X. (2025) ‘Pre-routing slack prediction based on graph attention network’, Automation, 6(2), 20.
9. Bhasker, J. and Chadha, R. (2009) Static Timing Analysis for Nanometer Designs: A Practical Approach. New York: Springer.
10. Clifford, G. D., Liu, C., Moody, B., et al. (2017) ‘AF classification from a short single lead ECG recording: the PhysioNet/Computing in Cardiology Challenge 2017’, Computing in Cardiology, 44.
11. He, K., Zhang, X., Ren, S. and Sun, J. (2016) ‘Deep residual learning for image recognition’, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
12. Hu, J., Shen, L. and Sun, G. (2018) ‘Squeeze-and-excitation networks’, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

13. Lin, T.-Y., Goyal, P., Girshick, R., He, K. and Dollár, P. (2017) 'Focal loss for dense object detection', Proceedings of the IEEE International Conference on Computer Vision (ICCV).

14. Vovk, V., Gammerman, A. and Shafer, G. (2005) Algorithmic Learning in a Random World. New York: Springer.

15. Ajayi, T., Chhabria, V. A., Fogaça, M., et al. (2019) 'Toward an open-source digital flow: first learnings from the OpenROAD project', Proceedings of the 56th Design Automation Conference (DAC).

16. SkyWater Technology and Google (2020) SkyWater SKY130 open source process design kit. https://github.com/google/skywater-pdk.

17. Paszke, A., et al. (2019) 'PyTorch: an imperative style, high-performance deep learning library', Advances in Neural Information Processing Systems (NeurIPS).

18. Wang, H., Zhang, Z., Xiong, H., Zou, D., Chen, Y. and Jin, H. (2024) 'GRAND: a graph neural network framework for improved diagnosis', IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 43(4), p. 1288 onwards.

19. Yang, T., He, G. and Cao, P. (2022) 'Pre-routing path delay estimation based on transformer and residual framework', 27th Asia and South Pacific Design Automation Conference (ASP-DAC).

20. Wang, Y., He, X., Zhao, R., Wang, Y., Liu, C. and Guo, Y. (2025) 'SI-aware wire timing prediction at pre-routing stage with multi-corner consideration', 30th Asia and South Pacific Design Automation Conference (ASP-DAC).

21. Wang, B., Shen, G., Li, D., Hao, J., Liu, W., Huang, Y., Wu, H., Lin, Y., Chen, G. and Heng, P. A. (2022) 'LHNN: lattice hypergraph neural network for VLSI congestion prediction', Proceedings of the 59th Design Automation Conference.

22. Mirhoseini, A., Goldie, A., et al. (2021) 'A graph placement methodology for fast chip design', Nature, 594, pp. 207–212.

23. Liu, Y., Ju, Z., Li, Z., et al. (2022) 'GraphPlanner: floorplanning with graph neural network', ACM Transactions on Design Automation of Electronic Systems.

24. Lu, Y.-C., Pentapati, S. and Lim, S. K. (2020) 'VLSI placement optimization using graph neural networks', Georgia Institute of Technology.

25. Gujarat Technological University (2018) Summer Internship, subject code 3170001, semester VII: syllabus and evaluation guidelines.